

Gisma  
University  
of Applied  
Sciences

Gisma University of Applied Sciences  
Department of Computer and Data Sciences

---

# Adversarial Robustness of Open-Source RAG Pipelines: A Cross-Model Benchmark of Prompt Injection, Knowledge Poisoning, and Multi-Turn Attacks Under EU AI Act Article 15

---

Simarjit Singh

GH1037512

Submitted in partial fulfillment of the requirements for the degree of  
MSc Data Science, AI, and Digital Business

Under supervision of  
Prof. Dr.-Ing. Loui Al Sardy

September 2026

# Abstract

RAG systems are now in production, but their robustness against adversarial manipulation remains largely untested, while Article 15 of the EU AI Act requires this kind of testing for sensitive and high-risk systems. This research benchmarks four open-source and open-weight LLMs, Llama-3.1-8B, Ministral-3-8B, Phi-4-mini, and Qwen3-8B, run across three corpora hotpot\_qa, ms\_marco, and nq\_open against three attack families (indirect prompt injection, PoisonedRAG, and Crescendo) and three defenses (instruction\_detection, spotlighting, and output\_filter). Later the pipeline drops nq\_open from every attack, defense and cross-corpus comparison because the corpus was leaking gold answers into its own retrieved context which led to every model scores reaching near roof on it regardless of proper retrieval or reasoning quality. Every reported metrics rests on paired statistical testing like McNemar’s test for paired model comparisons, Fisher’s test for independent model comparisons and Finally Holm Bonferroni correction within each family of hypotheses. Baselines for defended runs match the inference backend of the runs they compare against and every generation runs on real GPU hardware rather than any vague numbers. A dedicated verification process has been created to cross-checks these results. The three attacks later split along three different dimensions. Model identity drives injection success, Corpus identity drives PoisonedRAG Success, Crescendo produces no significant results across all models at all. A backend-confound investigation resulted in changing one defense’s headline number: output\_filter’s apparent effect, significant in 6 of 19 cells, dropped to 0 of 19 once the analysis controlled for a mismatch between the inference backends used in the baseline and defended runs. Three defenses close out the scorecard. output\_filter shows no significant effect on injection (0 of 19 cells) and a 0.0000 flag rate against PoisonedRAG across all 160 scored responses. Spotlighting had cut down the injection in 9 of 16 cells, but resulting in costing a mean  $\Delta F_1$  of  $-0.283$  against the no-attack baseline. Instruction\_detection reaches 0 of 16 significant cells at its capped  $n = 40$  samples, though its classifier mechanism had shown up behind all 10 confirmed blocks. These findings map onto Article 15(5) of the European Union’s Artificial Intelligence Act (Regulation (EU) 2024/1689) as a technical contribution: no harmonised standards yet exists for testing RAG-specific robustness under this regulation, so this benchmark represents what that testing looks like at real effect sizes. The mapping stops short of any legal claims or conformity-assessment claim and names its own gaps directly, including model-poisoning, confidentiality and sensitiv attacks that fall outside this research’s three attack families.

**Keywords:** *retrieval-augmented generation, adversarial robustness, prompt injection, knowledge poisoning, jailbreaking, EU AI Act Article 15*

# Declaration of Originality

I hereby declare that this dissertation is my own original work and has not been submitted for any other degree or professional qualification. All sources and materials used have been properly acknowledged and cited.

Simarjit Singh  
September 2026

---

# Acknowledgments

I would like to acknowledge my supervisor, Prof. Dr.-Ing. Loui Al Sardy, for their invaluable guidance, insightful feedback, and continuous support throughout this dissertation. I am also grateful to the Dissertation Team at Gisma University of Applied Sciences for their vluaable support and motivation during the course of this thesis. Special thanks to my family and friends for their implicit belief in me.

# Dedication

This dissertation is dedicated to my Professor Dr.Ing. Loui Al Sardy, my supervisor, for his timely guidance and response at every stage of this research and to my parents for their steady motivation which helped me to reach this point.

It is also dedicated, in a wider context, to the situation we find ourselves in – as the European Union works toward increased autonomy towards its AI, data and digital infrastructure. That same question, of who controls and secures the systems we increasingly depend on, is what grabs my attention as a reason to choose this topic.

# Contents

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                   | <b>1</b>  |
| 1.1      | Context and Motivation . . . . .                      | 1         |
| 1.2      | Problem Statement . . . . .                           | 1         |
| 1.3      | Research Questions . . . . .                          | 2         |
| 1.4      | Contributions . . . . .                               | 3         |
| 1.5      | Implementation and Reproducibility . . . . .          | 3         |
| 1.6      | Scope and Limitations . . . . .                       | 3         |
| 1.7      | Structure of the Dissertation . . . . .               | 4         |
| 1.8      | Conceptual Framework . . . . .                        | 5         |
| <b>2</b> | <b>Foundations</b>                                    | <b>6</b>  |
| 2.1      | Retrieval-Augmented Generation . . . . .              | 6         |
| 2.2      | Adversarial Robustness in Language Models . . . . .   | 7         |
| 2.3      | The Three Attack Families at a Glance . . . . .       | 7         |
| 2.4      | Where the Attacks Intervene in the Pipeline . . . . . | 8         |
| 2.5      | Regulatory Context: The EU AI Act . . . . .           | 8         |
| <b>3</b> | <b>Related Work</b>                                   | <b>10</b> |
| 3.1      | Indirect Prompt Injection . . . . .                   | 10        |
| 3.2      | Knowledge Poisoning of RAG Systems . . . . .          | 10        |
| 3.3      | Multi-Turn Jailbreaking . . . . .                     | 11        |
| 3.4      | Defenses . . . . .                                    | 11        |
| 3.5      | Reference Implementations Consulted . . . . .         | 12        |
| 3.6      | Gap This Thesis Addresses . . . . .                   | 13        |
| <b>4</b> | <b>Approach</b>                                       | <b>14</b> |
| 4.1      | System Architecture . . . . .                         | 14        |
| 4.2      | Target Models . . . . .                               | 15        |
| 4.3      | Statistical Testing Methodology . . . . .             | 16        |
| 4.4      | Attack Implementations . . . . .                      | 17        |
| 4.4.1    | Indirect Prompt Injection . . . . .                   | 17        |
| 4.4.2    | Knowledge Poisoning . . . . .                         | 17        |
| 4.4.3    | Multi-Turn Jailbreaking . . . . .                     | 17        |
| 4.5      | Defense Implementations . . . . .                     | 18        |
| 4.6      | Infrastructure and Engineering Challenges . . . . .   | 18        |
| <b>5</b> | <b>Evaluation and Results</b>                         | <b>22</b> |
| 5.1      | Experimental Setup . . . . .                          | 22        |
| 5.2      | Baseline Answer Quality . . . . .                     | 22        |
| 5.3      | The <code>nq_open</code> Exclusion . . . . .          | 23        |
| 5.4      | Attack 1: Indirect Prompt Injection . . . . .         | 24        |
| 5.5      | Attack 2: PoisonedRAG . . . . .                       | 26        |
| 5.6      | Attack 3: Crescendo . . . . .                         | 27        |
| 5.7      | Cross-Attack Synthesis . . . . .                      | 29        |

|          |                                                                 |           |
|----------|-----------------------------------------------------------------|-----------|
| 5.8      | Robustness Profile by Model . . . . .                           | 29        |
| 5.9      | Defense Evaluation and the Backend-Confound Discovery . . . . . | 30        |
| 5.10     | Defense-Utility Trade-off . . . . .                             | 33        |
| 5.11     | Efficiency: A Named Scope Boundary . . . . .                    | 34        |
| 5.12     | Comparison to Related Benchmarks . . . . .                      | 34        |
| 5.13     | Mapping to EU AI Act Article 15 . . . . .                       | 34        |
| 5.14     | Implications for Practitioners . . . . .                        | 35        |
| 5.15     | Answering the Research Questions . . . . .                      | 36        |
| <b>6</b> | <b>Conclusion</b>                                               | <b>37</b> |
| 6.1      | Summary of Contributions . . . . .                              | 37        |
| 6.2      | Practical Implications . . . . .                                | 38        |
| 6.3      | Limitations . . . . .                                           | 39        |
| 6.4      | Future Work . . . . .                                           | 40        |
|          | <b>References</b>                                               | <b>45</b> |
|          | <b>Appendices</b>                                               | <b>46</b> |
|          | <b>List of Figures</b>                                          | <b>49</b> |
|          | <b>List of Tables</b>                                           | <b>50</b> |

# Chapter 1

## Introduction

### 1.1 Context and Motivation

Retrieval-Augmented Generation (RAG) has become a default way to give large language models access to information beyond their initial training data. Instead of relying solely on parametric knowledge, a RAG-based model retrieves relevant data from an external corpus at inference time and adjusts its responses based on that retrieved context. This pattern is now used in a variety of production systems across search, customer support, and enterprise knowledge tools: enterprise assistants that answer questions against internal documentation, customer-support chatbots that ground replies in a company's own knowledge base, and consumer-facing assistants that cite sources for factual claims rather than relying purely on what a model memorized during training. The appeal is straightforward. Parametric knowledge freezes at a model's training cutoff and cannot reflect anything that happened afterward, and a model reasoning purely from its own weights has no built-in mechanism to point to a specific source for a claim it makes. Grounding generation in retrieved text addresses both problems at once, at the cost of introducing a new component into the pipeline that an adversary can target: the retrieval corpus itself.

Retrieval has also opened a new attack pattern. An adversary who can influence the retrieved content or the conversations built around it can manipulate the model's output without ever touching its model weights, without needing any access to the model's training process, and often without the system's operators noticing anything unusual about the request itself. The European Union's Artificial Intelligence Act (Regulation (EU) 2024/1689) has given this question a regulatory importance: Article 15 requires high-risk AI systems to display "an appropriate level of accuracy, robustness, and cybersecurity" and to be resilient against attempts to exploit system vulnerabilities. This dissertation treats Article 15(5) as a technical specification to test against, not a legal claim to argue, a distinction maintained throughout the rest of the dissertation.

### 1.2 Problem Statement

Three distinct mechanisms exist for manipulating a RAG system's output through its retrieval layer, and treating them as variations on one idea rather than three separate threats is itself a methodological risk this dissertation avoids. The first, planting a malicious instruction inside retrieved text, known as indirect prompt injection Greshake et al. (2023), exploits the fact that a model has no reliable way to distinguish an instruction written by its system prompt from one embedded in a document it retrieves. The second, planting a false fact into the model's corpus itself, known as knowledge poisoning Zou et al. (2025), targets the corpus rather than the model's instruction-following behavior: an adversary inserts a small number of crafted documents into the knowledge base such that, for a chosen query, the poisoned documents are retrieved and steer the model toward a specific false answer. The third, manipulating the model

across multiple conversational turns, exploiting its conversational memory state rather than any single retrieved document Russinovich et al. (2025), escalates a benign-seeming conversation gradually toward an output the model would refuse if asked directly in a single turn. Each targets a genuine vulnerability in the pipeline rather than a variation on a single idea, so a defense effective against one offers no guarantee against another.

Existing RAG security benchmarks tend to cover a broader set of attacks within a single attack category or one narrow slice of the threat surface, rather than all three families side by side under one shared, controlled environment. SafeRAG Liang et al. (2025) evaluates four RAG-specific attack tasks (noise, conflict, toxicity, denial-of-service) across fourteen RAG components, but its safety framework has no multi-turn conversational attacks, and it does not test or evaluate the same four target models this research studies. RSB Zhang et al. (2025) benchmarks thirteen poisoning attacks and seven defenses at far greater scale than this research’s single poisoning method, but covers poisoning exclusively, with no injection or multi-turn attack family at all. RSB remains an arXiv preprint at the time of writing this, and no other trusted version has emerged despite deep searches across the web. This dissertation includes it here as an explicit exception to its peer-review sourcing standard because it was one of three benchmark works confirmed directly with the supervisor. Crucially, its own primary finding shows that its strongest defense reduces poisoning success but at an accuracy drop exceeding 20% in most settings, which parallels this dissertation’s own finding that spotlighting is the only defense with a confirmed real effect on injection, at a mean  $\Delta F_1$  of  $-0.283$ . Hidden-in-Plain-Text Guo and Wei (2026) reports attack-success confidence intervals for indirect prompt injection specifically, but is scoped to web-carrier delivery mechanisms and doesn’t cover knowledge poisoning or multi-turn jailbreaking. No existing work to this dissertation’s knowledge evaluates all three attack families against the same set of models under a single backend-controlled harness; the backend-confound discovery detailed in Chapter 4 is itself a methodological consideration most of this literature doesn’t appear to control for.

## 1.3 Research Questions

This dissertation explores two research questions:

- **RQ1:** How vulnerable are small-to-mid-size (4–12B parameters) open-source, open-weight, instruction-tuned language models to indirect prompt injection, knowledge poisoning and multi-turn adversarial manipulation when deployed in a retrieval-augmented generation pipeline?
- **RQ2:** How effective are practical, low-cost defenses at reducing attack success and at what cost to the system’s underlying answer utility?

A third, narrower question follows from the first two and runs through the closing chapters of this dissertation rather than standing on its own: how do the answers to RQ1 and RQ2 map onto Article 15(5)’s specific vulnerability classes, and where does that mapping expose gaps in what a benchmark of this kind can and cannot demonstrate? This third question is framed deliberately as an open mapping exercise rather than a claim to be proven, since the goal of Chapter 5’s regulatory discussion is honesty about where the benchmark’s evidence is strong and where it is silent, not forcing every result into a compliance verdict it cannot support.

## 1.4 Contributions

This dissertation contributes:

1. A reproducible, statistically rigorous, real-hardware benchmark spanning four open-source language models, two retrieval corpora, three attack families and three defenses, built so that every reported number traces back to a specific, re-runnable script and a verified result file rather than a one-off measurement.
2. A documented account of an inference-backend confound discovered mid-analysis, which overturned two of the three defenses’ apparent headline effects once corrected, offering a citable methodological warning for anyone building a similar evaluation: a defense’s apparent effect can be, in whole or in part, an artifact of which inference backend its baseline happened to run on, and this is detectable only by deliberately building a backend-matched comparison.
3. A mapping of these results onto Article 15(5)’s vulnerability classes, offered as a concrete technical contribution to a compliance-testing gap that currently has no harmonised standard, together with an explicit account of which parts of Article 15 this benchmark’s design cannot speak to.

The central empirical finding running through all three contributions is that robustness is not one factor. Model identity dominates injection success. Corpus identity dominates knowledge-poisoning success. The multi-turn attack produces no statistically significant ranking across models at all. A benchmark, defense, or compliance argument built around one blended notion of robustness would miss this structure entirely, and would risk recommending a defense calibrated to the wrong axis of vulnerability.

## 1.5 Implementation and Reproducibility

This dissertation evaluates four open-source, instruction-tuned language models: Llama-3.1-8B-Instruct Grattafiori et al. (2024), Qwen3-8B Yang et al. (2025), Phi-4-mini-instruct Abouelenin et al. (2025), and Ministral-3-8B-Instruct-2512 Mistral AI (2025). These four models were selected against five explicit requirements, detailed further in Chapter 4: a permissive open license, self-hostable weights, instruction tuning, a parameter count in the roughly 4–12B range, and a dense, non-mixture-of-experts architecture. Every result rests on real generations run on real GPU hardware RunPod (2024), not simulated or estimated output, a distinction that matters given how much of the adjacent literature relies on smaller samples or proxy measurements. The full implementation, an open-source Python framework, is available at <https://github.com/Simarjit1303/rag-adversarial-robustness> Singh (2026); it satisfies this module’s technical-contribution requirement and lets another researcher reproduce, extend, or challenge any result reported here directly against the same code and data.

## 1.6 Scope and Limitations

This dissertation uses two retrieval corpora, HotpotQA Yang et al. (2018) and MS MARCO Nguyen et al. (2016), across every comparison. It also evaluated a third corpus, Natural Questions Open Kwiatkowski et al. (2019), and excluded it after it became structurally unsuitable for testing retrieval-dependent robustness: its passage text is, by construction, the gold answer itself, a property detailed with full evidence in Chapter 5. The supervisor confirmed this decision as a settled scope choice. This dissertation excludes model poisoning and confidential-

ity attacks from its scope: every attack it studies operates at inference time, and none touch training data or model weights, a boundary worth stating plainly rather than stretching any of the three studied attacks to cover ground they were never designed for. It does not determine whether any specific deployed RAG system counts as high-risk under Annex III of the AI Act; that determination depends on deployment context outside this benchmark’s scope, and this dissertation makes no claim about it in either direction.

## 1.7 Structure of the Dissertation

This chapter has presented the motivation, problem, research questions, and contributions towards this research. Figure 1.1 outlines the chapters that follow.

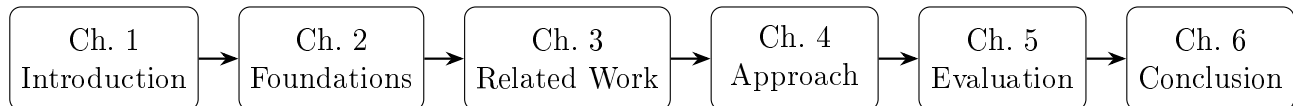


Figure 1.1: Structure of the dissertation, chapters in reading order.

As Figure 1.1 shows, five chapters follow after this one. Chapter 2 (Foundations) establishes the technical background needed to follow the rest of the dissertation: retrieval- augmented generation itself, the model-versus-context distinction that organizes this dissertation’s whole attack taxonomy, and the regulatory context this dissertation tests against. Chapter 3 (Related Work) surveys the existing literature on RAG-specific adversarial attacks and defenses, organized by attack family and defense mechanism; under the framework set by the university’s CDS structure, this chapter serves as the dissertation’s literature review. Chapter 4 (Approach) describes the benchmark’s design and implementation in detail, including the infrastructure it runs on and the real engineering failures encountered while building it. Chapter 5 (Evaluation and Results) presents the full statistical analysis across all three attack families and three defenses, and maps them onto Article 15. Chapter 6 (Conclusion) summarizes the findings, states their limitations honestly, and closes with a discussion of what this benchmark contributes to, and cannot resolve about, the compliance-testing question this dissertation opened with.

## 1.8 Conceptual Framework

Figure 1.2 maps how this dissertation’s core constructs relate to one another, the research design itself, not the chapter order Figure 1.1 shows or the operational pipeline Chapter 4 describes

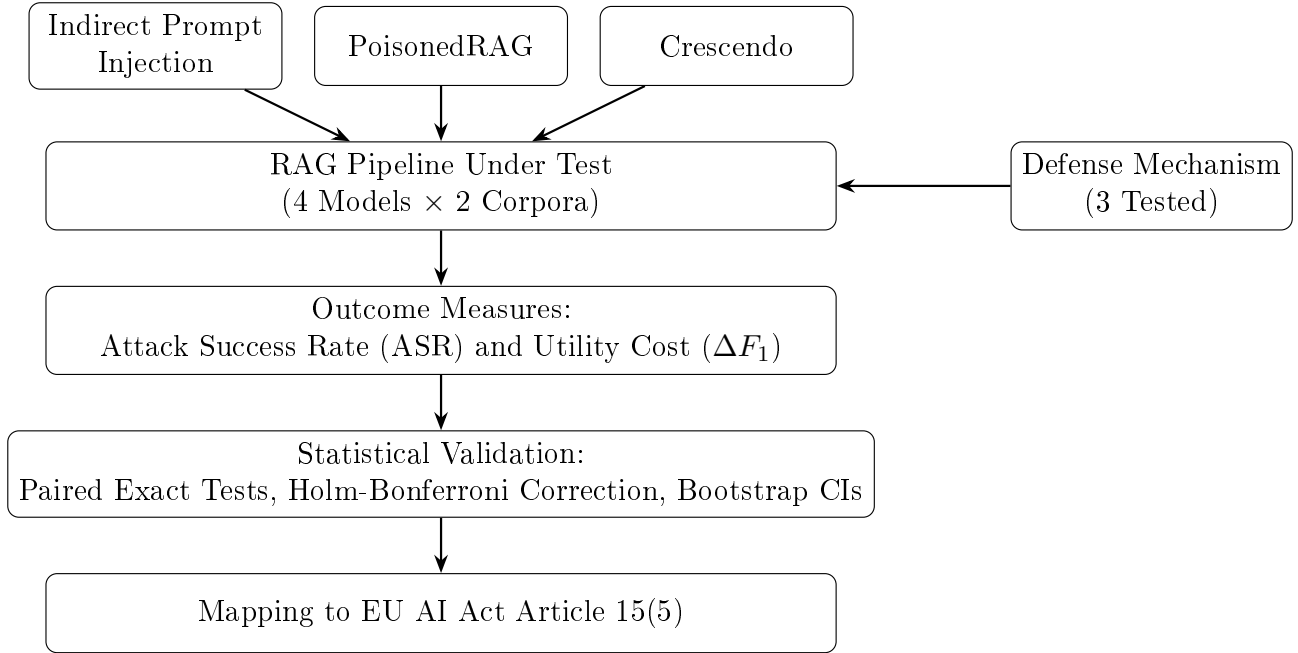


Figure 1.2: Conceptual framework: how attacks, defenses, and outcomes connect through this dissertation’s statistical methodology to its Article 15 mapping.

Three attack families target the RAG pipeline from the top of the figure, each exploiting a structurally distinct surface, as Chapter 2 details: indirect prompt injection manipulates retrieved content, PoisonedRAG corrupts the retrieval index itself and Crescendo exploits multi-turn conversational state independently of retrieval altogether. A tested defense mechanism intervenes on the pipeline from the side, positioned deliberately as a moderating influence on the attack’s effect rather than a fourth attack surface. The pipeline’s response under each attack-defense combination produces two outcome measures, attack success rate and utility cost, and neither is reported as a raw number on its own. Every reported figure passes through a statistical validation stage first, paired exact testing appropriate to the comparison structure, Holm-Bonferroni correction within each hypothesis family and bootstrap confidence intervals for utility-cost estimates, detailed fully in Chapter 4, before it counts as a finding. Only validated findings feed into the final stage, this dissertation’s mapping onto Article 15(5), which is why that mapping in Chapter 5 can speak to the compliance-testing question with statistically confirmed evidence rather than figures no correction has touched.

# Chapter 2

## Foundations

### 2.1 Retrieval-Augmented Generation

Retrieval-augmented generation (RAG) grounds a language model’s output in text retrieved from an external corpus at inference time rather than relying solely on knowledge encoded in the model’s parameters during training. This matters for two related reasons: parametric knowledge freezes at the training cutoff and cannot reflect anything after it, and a model reasoning purely from its own weights has no mechanism to point to a specific source for a claim it makes. A model asked a factual question can produce a fluent, confident answer regardless of whether that answer is actually correct, and without a retrieved passage backing it, there is no way to check the claim against anything other than the model’s own internal state. Grounding generation in retrieved text addresses both problems at once, at the cost of introducing a new component into the pipeline, the corpus itself, that an adversary can try to influence.

A RAG pipeline follows a consistent retrieve-then-generate pattern, illustrated in Figure 2.1. A sentence embedding model first encodes the user query into a dense vector Reimers and Gurevych (2019); Song et al. (2020). An approximate nearest-neighbour search then compares this vector against a pre-built index of the corpus’s passages Johnson et al. (2021) and returns the top- $k$  most similar passages. The pipeline integrates these passages with the original query to form a single prompt and passes it to the language model, served in this benchmark through a high-throughput inference engine Kwon et al. (2023), to generate an answer. Each of these three stages, embedding, indexing, and serving, is a separate piece of infrastructure with its own failure modes, and this dissertation’s later chapters return to that fact directly: an attack does not need to compromise the language model itself to change the system’s output, it only needs to influence what the retrieval stage hands the model.

Table 2.1 summarizes the specific implementation of each pipeline stage used in this benchmark.

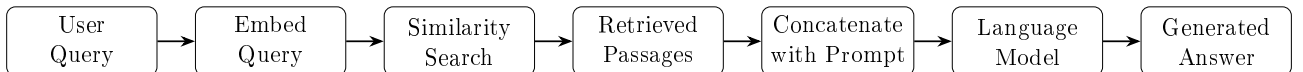


Figure 2.1: The retrieve-then-generate pipeline used throughout this dissertation.

| Component        | Implementation    | Source                                          |
|------------------|-------------------|-------------------------------------------------|
| Embedding model  | all-mpnet-base-v2 | Reimers and Gurevych (2019); Song et al. (2020) |
| Vector index     | FAISS             | Johnson et al. (2021)                           |
| Inference engine | vLLM              | Kwon et al. (2023)                              |

Table 2.1: Core RAG pipeline components used in this benchmark.

## 2.2 Adversarial Robustness in Language Models

Robustness research in this space typically reports an attack success rate (ASR): the proportion of attempts in which an adversary achieves their stated objective, whether that objective is eliciting a specific answer, bypassing a safety filter, or causing a refusal. ASR is a useful summary metric precisely because it abstracts away from any single attack’s mechanics, but that abstraction can also obscure a distinction that matters a great deal for a RAG system specifically: an attack can target the model itself, or it can target the context the model is given.

An attack on the model, commonly called jailbreaking, manipulates the conversation or the prompt directly, without needing the model to be retrieving anything at all. An attack on the context instead exploits the retrieval step: it plants something in the corpus or the retrieved passages themselves, so that the model behaves as attacked without any direct manipulation of its own weights or its own conversational input. The practical consequence of this split is significant for anyone building a defense. A safeguard that inspects the user’s conversation for suspicious phrasing does nothing to stop a poisoned passage sitting quietly in the corpus, and a safeguard that filters retrieved content does nothing to stop a multi-turn escalation that never touches retrieval at all. This model-versus-context distinction is the organising idea behind this dissertation’s entire attack taxonomy, and it is the reason the three attack families studied here receive three separate treatments in Chapter 5 rather than being collapsed into a single blended ASR figure: a defense that works against one surface carries no logical guarantee against the other.

## 2.3 The Three Attack Families at a Glance

**Indirect prompt injection** Greshake et al. (2023) plants a malicious instruction inside content the model will later retrieve. The model has no reliable way to distinguish an instruction written by its system prompt from one embedded in a retrieved passage, so a sufficiently convincing injected instruction can redirect the model’s behaviour even though the user’s own query never mentioned it. The attack requires no access to the corpus’s construction process, only the ability to get one crafted document indexed alongside legitimate content, which is a low bar for any system that ingests documents from a source an adversary can write to.

**Knowledge poisoning** Zou et al. (2025) instead targets the corpus itself rather than any single retrieved passage’s instructions. An adversary inserts a small number of crafted documents into the knowledge base such that, for a chosen query, the poisoned documents are retrieved and cause the model to state a specific false answer as though it were grounded fact. Unlike injection, this attack does not rely on the model following an embedded instruction at all; it relies on the model doing exactly what it is designed to do, trusting retrieved content, against content that was never legitimate in the first place.

**Multi-turn jailbreaking** Russinovich et al. (2025) operates differently again: rather than attacking a single retrieved document, it exploits the model’s growing conversational state across several turns, gradually escalating a benign-seeming conversation toward a harmful or policy-violating output that the model would have refused if asked directly in a single turn. This attack touches neither the corpus nor any retrieved passage, which is precisely why a defense built around inspecting retrieved content offers it no resistance at all.

## 2.4 Where the Attacks Intervene in the Pipeline

Returning to Figure 2.1 makes the model-versus-context distinction easier to see. Indirect prompt injection intervenes late in the pipeline, at the point where retrieved passages are concatenated with the query to form a single prompt: the malicious instruction rides inside a passage that has already survived retrieval, and by the time it reaches the language model it is indistinguishable, in form, from any other piece of retrieved text. The attack does nothing to the embedding step, the similarity search, or the index itself; it only needs one crafted document to already be sitting in the corpus, waiting to be picked up by an ordinary query.

Knowledge poisoning intervenes earlier and more fundamentally: it corrupts the index that the similarity-search stage draws from, before anyone ever issues a specific query. Injection waits for retrieval to happen and then exploits what gets concatenated; poisoning shapes what is available to be retrieved in the first place. This distinction carries directly into this dissertation's results, detailed fully in Chapter 5: injection's success turns on which model receives the crafted instruction, since different models parse and act on embedded instructions differently, while poisoning's success hinges on which corpus the poisoned passage competes against for retrieval, since a corpus with denser, more redundant coverage of a topic makes it harder for one poisoned document to win the top- $k$  ranking.

Multi-turn jailbreaking does not map onto a single stage of Figure 2.1 at all. The attack operates across repeated invocations of the whole pipeline rather than at any one node inside it: each turn is its own retrieve-then-generate cycle, and the vulnerability lies in the conversational history accumulating around those cycles, not in the retrieval or generation of any single turn. A defense implemented purely as a filter on retrieved passages, wherever it sits in the diagram, has nothing to inspect for this attack, since nothing about a Crescendo-style escalation depends on what the corpus contains.

## 2.5 Regulatory Context: The EU AI Act

The European Union's Artificial Intelligence Act, Regulation (EU) 2024/1689,<sup>1</sup> adopted 13 June 2024, sets binding requirements for high-risk AI systems. Article 15 requires such systems to achieve an appropriate level of accuracy, robustness, and cybersecurity, and, more specifically under Article 15(5), to remain resilient against attempts by unauthorised third parties to exploit system vulnerabilities. The Article's own text names several vulnerability classes this resilience requirement targets: data poisoning, model poisoning, adversarial examples, and confidentiality attacks, each named as an illustrative example of the kind of exploitation the general clause is meant to catch, not as an exhaustive list closed to anything else. The Article also qualifies its own cybersecurity requirement directly: the technical solutions used to meet it must be "appropriate to the relevant circumstances and the risks," a proportionality standard, not a fixed technical bar, which leaves open exactly the kind of empirical question this dissertation investigates: what counts as an appropriate mitigation, in practice, against a specific attack family.

At the technical level this dissertation operates at, these vulnerability classes correspond closely to the attack families introduced above. Knowledge poisoning maps onto the Article's data-poisoning class directly, since both describe an adversary inserting false material into a system's

---

<sup>1</sup>This dissertation verified this citation specifically against a 2026 amendment, the Digital Omnibus, Regulation (EU) 2026/1744, which could otherwise be mistaken for the AI Act itself given its proximity in both subject matter and numbering; Article 15's substantive requirements discussed throughout this dissertation belong to Regulation (EU) 2024/1689, not its later amendment.

knowledge source. Indirect prompt injection functions as an adversarial-examples attack against the model's input: an adversarial example is deliberately crafted to trigger a mistake, not something that arises by chance, and an injected instruction is exactly that, a targeted construction aimed at making the model behave incorrectly. Multi-turn jailbreaking is not named explicitly in the Article's illustrative list, a point this dissertation returns to directly in Chapter 5, but it plausibly falls under the Article's general resilience clause regardless, since escalating a conversation to bypass a system's safeguards is a form of exploiting a system vulnerability even when that vulnerability is conversational rather than architectural. Whether a specific mitigation counts as "appropriate" under the Article's own proportionality standard is an empirical question this dissertation's results speak to directly: a mitigation's appropriateness is hard to judge without first measuring what it actually does and does not achieve against each attack family. This dissertation treats Article 15(5) throughout as a technical specification to test the benchmark's findings against, not as a legal claim about any deployed system's regulatory status, a distinction established in Chapter 1 and maintained here.

# Chapter 3

## Related Work

### 3.1 Indirect Prompt Injection

Greshake et al. (2023) established indirect prompt injection as a distinct attack class, demonstrating that real-world LLM-integrated applications could be compromised by instructions embedded in content the model retrieves or processes, not in the user’s own input. Their work was the first to frame this as a structurally different threat from a jailbreak, since the attacker never interacts with the system at all; the malicious content simply waits in a document, a webpage, or an email for the model to encounter it during normal operation. Their evaluation demonstrates the attack qualitatively, through real-world case studies against deployed integrations, rather than the quantified, cross-model attack-success-rate comparison this dissertation’s own methodology builds on those foundations to produce. Liu et al. (2024) built on this by formalising prompt injection attacks and defenses into a shared benchmarking methodology, published at USENIX Security 2024, with explicit attack-success metrics that later work could compare against directly rather than each paper inventing its own measurement convention. This dissertation’s own attack-success-rate methodology, detailed in Chapter 4, follows a comparable measurement approach, applied specifically to the RAG retrieval surface rather than to the general LLM-integrated applications Liu et al. study, a narrower scope chosen deliberately so that every result in this dissertation can be attributed to the retrieval pipeline itself rather than to some other integration point. Zhan et al. (2024) extends the injection threat model to tool-integrated LLM agents, where an injected instruction can trigger unintended tool calls, not just an unintended text response. This is adjacent context for the threat landscape, not something this dissertation builds on directly, since the benchmark here targets a plain retrieve-then-generate pipeline without agentic tool use; an injected instruction in this dissertation’s setting can only ever change what text the model produces, never trigger an external action.

### 3.2 Knowledge Poisoning of RAG Systems

Zou et al. (2025) introduced PoisonedRAG, the knowledge-poisoning attack implemented directly in this dissertation: an adversary inserts a small number of crafted documents into the knowledge base such that, for a chosen query, the poisoned documents are retrieved and guide the model toward a specific false answer. The attack’s original formulation splits each poisoned document into a retrieval component, designed to win the similarity search, and a generation component, designed to steer the model’s answer once retrieved, a two-part structure this dissertation’s own implementation preserves. The original paper evaluates the attack in isolation, with no defended condition and no backend-controlled comparison; testing it against three structurally distinct defenses under conditions that isolate an infrastructure confound is this dissertation’s own extension of that design. Moradi et al. (2026) proposes a 2026 defense benchmarked directly against PoisonedRAG, using cross-encoder activation signals to detect poisoned passages before they reach the generation stage; its continued use of PoisonedRAG as

the reference attack, more than two years after the original paper, is evidence that the attack remains the field’s active benchmark for knowledge poisoning rather than having been superseded by something newer. That defense also operates on internal cross-encoder activation signals, requiring access to model internals a deployer may not have; all three defenses tested here work externally and model-agnostically instead, a more restrictive but more broadly deployable class of mitigation.

### 3.3 Multi-Turn Jailbreaking

Russinovich et al. (2025) introduced Crescendo, the multi-turn jailbreak attack implemented directly in this dissertation, which escalates a benign-seeming conversation gradually across several turns toward a harmful or policy-violating output that a model would refuse if asked directly in a single turn. The attack’s core insight is that a model’s safety training is calibrated against single-turn requests, so a goal broken into a sequence of individually innocuous-looking steps can slip past defenses tuned for the more obvious, one-shot version of the same request. Russinovich et al.’s own tests target a different set of deployed conversational systems; running the same attack against the four open-weight models already benchmarked here against two other attack families is what lets this dissertation compare all three attacks head to head in Chapter 5, something a single-system evaluation was never designed to support. The behaviour pool used to instantiate Crescendo’s escalation targets in this dissertation draws on two established sources: Chao et al. (2024)’s JBB-Behaviors, contributing 100 behaviours and 20% of the stratified pool, and Mazeika et al. (2024)’s HarmBench, contributing 510 behaviours and the remaining 80%. Combining the two sources this way follows current practice in the guardrail literature, where a single behaviour set is generally considered too narrow to represent the diversity of real harmful-request categories on its own. Nakka and Saxena (2026) is contemporaneous 2026 work, published at EACL 2026, that jailbreaks aligned models through bitstream camouflage rather than conversational escalation, a fundamentally different delivery channel that this dissertation’s Crescendo implementation does not test and does not claim to cover.

### 3.4 Defenses

Hines et al. (2024) introduced spotlighting, the defense implemented directly in this dissertation, in its encoding-mode variant, which the original paper reports as the strongest configuration against indirect prompt injection, closer to a 0% attack success rate than the paper’s alternative datamarking variant achieves. The original spotlighting evaluation covers injection alone, outside a retrieval pipeline; testing it under genuine retrieval noise and against a second, structurally distinct attack family is where this dissertation’s own severe utility-cost finding for spotlighting first surfaces, a result their evaluation had no occasion to reveal. For the instruction-detection defense, this dissertation uses a classifier fine-tuned from the DeBERTaV3 architecture (He et al., 2023), published at ICLR 2023, specifically Protect AI (2024)’s released `deberta-v3-base-prompt-injection-v2` model. The closest peer-reviewed comparator for this style of defense is Liu et al. (2025)’s DataSentinel, published at IEEE Symposium on Security and Privacy 2025, a game-theoretic detection method that models the interaction between attacker and defender explicitly rather than training a static classifier; a different, pre-trained classifier was used in this dissertation in practice, for the practical reason that fine-tuning a comparable detector from scratch was outside this project’s compute and time budget. The two approaches trade off differently: DataSentinel’s game-theoretic training against a simulated attacker offers stronger in-principle robustness at the cost of a training pipeline most labs do not already have in place, while reusing an existing pretrained classifier sacrifices some of that

robustness for a setup any researcher can reproduce without retraining anything. The output-filter defense uses Meta AI (2025)’s Llama Guard 4, cited here as a released model rather than a peer-reviewed study. Finally, Cui et al. (2025)’s OR-Bench, from ICML 2025, and Röttger et al. (2024)’s XSTest, from NAACL 2024, provide the over-refusal and false-positive benchmarking used to evaluate whether these defenses reject legitimate, benign queries, a concern distinct from, but as important as, their effectiveness against genuine attacks. XSTest’s 450 prompts, 250 marked safe and 200 marked unsafe, are paired in this dissertation with OR-Bench’s harder-but-still-safe prompt set specifically because XSTest’s own safe-prompt coverage is comparatively narrow; the two together give a broader picture of whether a defense over-triggers on ordinary, non-malicious input than either benchmark provides alone.

Figure 3.1 groups the sources discussed above by which part of the threat surface they address.

| Injection             | Poisoning           | Multi-Turn               | Defenses             |
|-----------------------|---------------------|--------------------------|----------------------|
| Greshake et al., 2023 | Zou et al., 2025    | Russinovich et al., 2025 | Hines et al., 2024   |
| Liu et al., 2024      | Moradi et al., 2026 | Chao et al., 2024        | He et al., 2023      |
| Zhan et al., 2024     |                     | Mazeika et al., 2024     | Liu et al., 2025     |
|                       |                     | Nakka & Saxena, 2026     | Meta AI, 2025        |
|                       |                     |                          | Cui et al., 2025     |
|                       |                     |                          | Röttger et al., 2024 |

Figure 3.1: Literature landscape: sources grouped by attack family and defense mechanism.

### 3.5 Reference Implementations Consulted

Beyond the published papers discussed above, this dissertation’s own attack implementations are original code, not adaptations of an existing codebase. A repository-wide search of the source tree and its full commit history found no verbatim external code, no vendored files, and no third-party license headers anywhere in the attack modules. This dissertation nonetheless consulted two external implementations during development, not as a source of code to copy, but to check methodological fidelity against each paper’s own description of its algorithm.<sup>1</sup> For PoisonedRAG, the paper’s own official reference implementation, [sleepeer/PoisonedRAG](#) on GitHub, informed this dissertation’s implementation of the attack’s two-part poisoned-document structure. For Crescendo, Microsoft’s PyRIT toolkit provides a `CrescendoOrchestrator` that serves as the reference implementation of the paper’s escalation-and-backtrack strategy; this dissertation originally consulted it at its earlier repository location, [Azure/PyRIT](#), since archived and redirected to its current canonical location, [microsoft/PyRIT](#). Consulting a paper’s own reference implementation where one exists is standard practice when reimplementing a published attack, since it reduces the risk of a subtle

<sup>1</sup>Neither repository serves as a source of copied code; this dissertation discloses both as implementations it read for methodological correctness while reproducing each paper’s described algorithm, consistent with standard practice for reimplementing a published attack.

misreading of the paper’s prose diverging from what the original authors actually built and evaluated.

### 3.6 Gap This Thesis Addresses

Two gaps in the existing literature motivate this dissertation’s design, summarized in Table 3.1 against the three named comparator benchmarks introduced in Chapter 1.

First, no existing work evaluates all three attack families against the same set of models under a single, backend-controlled harness. SafeRAG (Liang et al., 2025) covers a broader attack taxonomy but omits multi-turn attacks and does not test this dissertation’s four target models. RSB (Zhang et al., 2025) covers poisoning at far greater scale, but exclusively, with no injection or multi-turn family in its design at all. Hidden-in-Plain-Text (Guo and Wei, 2026) covers injection specifically but not poisoning or multi-turn jailbreaking. None of the three, individually or together, offers what this dissertation’s design provides: the same four models, the same infrastructure, and the same statistical methodology applied across all three attack families at once, which is precisely what makes a genuine cross-attack comparison, like the one presented in Chapter 5, possible in the first place. The backend-confound discovery detailed in Chapters 4 and 5, where an inference-backend mismatch between baseline and defended runs produced a misleading defense result until corrected, is itself evidence that this kind of infrastructure-level control is a methodological consideration most of this literature does not appear to address explicitly; a benchmark that never tests multiple defenses under a matched infrastructure setup has no occasion to discover a confound of this particular kind.

Second, no existing work bridges the gap between practical RAG robustness evaluations and concrete compliance-testing requirements under Article 15 of the EU AI Act. SafeRAG, RSB, and Hidden-in-Plain-Text were all built as security benchmarks in their own right, and none frame their results against a specific regulatory text. This dissertation’s mapping of its own findings onto Article 15(5)’s vulnerability classes, developed fully in Chapters 5 and 6, is offered as a first step toward closing that gap, rather than a claim that a finalised, harmonised technical standard for this kind of testing yet exists; the mapping is explicitly a technical exercise applied after the fact to results that were designed as a security benchmark first, not a compliance-testing tool designed from the ground up.

| Dimension                | SafeRAG | RSB | Hidden-in-Plain-Text | This Dissertation |
|--------------------------|---------|-----|----------------------|-------------------|
| Injection                | ✓       |     | ✓                    | ✓                 |
| Poisoning                | ✓       | ✓   |                      | ✓                 |
| Multi-turn               |         |     |                      | ✓                 |
| Shared 4-model harness   |         |     |                      | ✓                 |
| Backend-confound control |         |     |                      | ✓                 |
| Article 15 mapping       |         |     |                      | ✓                 |

Table 3.1: Positioning this dissertation against its three named comparator benchmarks.

# Chapter 4

## Approach

### 4.1 System Architecture

Figure 4.1 shows the operational pipeline every sweep in this benchmark runs on. A push to the project’s GitHub repository triggers a CI build that pushes a container image to GHCR; a RunPod A100 GPU pod then pulls that image, configured via environment variables selecting the target model, corpus, attack, and defense for that run. The pod executes the harness’s model-loading, retrieval-index, attack, and defense modules against a persistent network volume, writes results as JSONL and CSV, and a watcher process verifies successful completion before terminating the pod, rather than terminating on any exit, successful or not, which would risk destroying an in-progress run’s partial output. The researcher then commits verified results to the repository manually, as a permanent record.

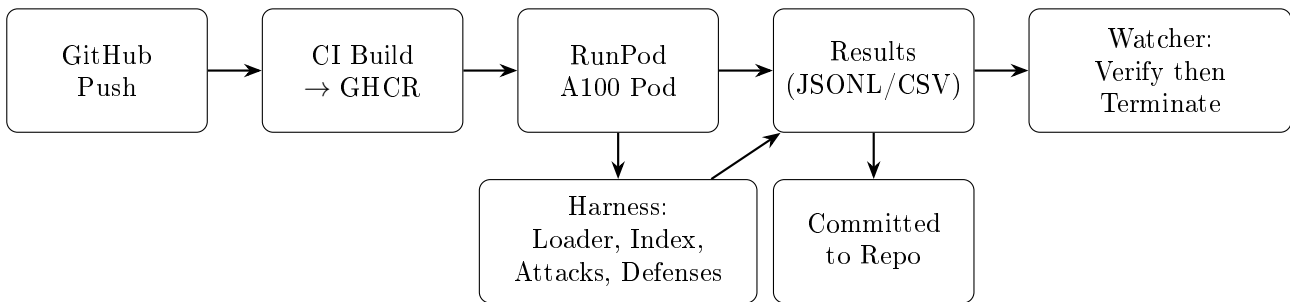


Figure 4.1: Operational pipeline: from a GitHub push to a verified, committed result.

The watcher’s self-terminating design matters more than it might appear at first glance. A GPU pod left running idle after a sweep finishes incurs real cost with nothing to show for it, and a pod that terminates automatically on any exit, whether the sweep actually succeeded or crashed halfway through, risks silently discarding a partially completed run’s real, if incomplete, data. RunPod’s start-command override runs independently of any attached SSH session, so a long sweep survives a terminal disconnect by construction; the watcher then verifies success before terminating, and idles rather than terminates on any ambiguous outcome, accepting a wasted compute cost as the safer failure mode compared to losing data outright. An earlier deployment path through Azure Container Apps Jobs was abandoned during development after a billing incident and a failed institutional verification step; the CI workflow steps for that path still exist and still run on every push, but fail harmlessly against a service that was never actually provisioned, since the GHCR build step above them succeeds and completes independently, which is all the RunPod deployment path actually depends on.

Two judge models score attack outcomes throughout this benchmark: Llama-Guard-4-12B (Meta AI, 2025) as the primary judge, an upgrade from an initially planned Llama-Guard-3-8B made at effectively zero switching cost, since the judge harness had not yet been built at

the point the newer model became available, and Qwen3Guard-Gen-8B as a secondary judge, independently validated as a top-performing open guard model at an ICLR 2026 workshop.

This benchmark runs every generation through one of two inference engines, selected per sweep via a single environment variable: HuggingFace’s `transformers` library (Hugging Face, 2016), the default, used for every Phase 3 defended run and for two of the three attacks’ Phase 2 baselines, and vLLM, used for the Phase 1 baseline sweep and the injection attack’s Phase 2 baseline. These two engines are not numerically interchangeable on the same hardware. vLLM auto-selects bf16 precision on the project’s A100 GPUs and auto-detects Ministral-3-8B’s native fp8 checkpoint through its own optimized kernel path, while the plain HuggingFace path dequantizes that same checkpoint to bf16 regardless of GPU generation. This mismatch is the concrete mechanical root cause behind the inference-backend confound discovered during defense evaluation and detailed fully in Chapter 5, where comparing a vLLM-run baseline against an HuggingFace-run defended condition silently introduced a second, uncontrolled variable alongside the defense itself.

## 4.2 Target Models

Table 4.1 lists the four target models this dissertation benchmarks. This dissertation cites all four as released models or technical reports, not peer-reviewed studies, since none underwent academic peer review prior to release.

| Model                        | Source     | Citation                   |
|------------------------------|------------|----------------------------|
| Llama-3.1-8B-Instruct        | Meta AI    | (Grattafiori et al., 2024) |
| Qwen3-8B                     | Alibaba    | (Yang et al., 2025)        |
| Phi-4-mini-instruct          | Microsoft  | (Abouelenin et al., 2025)  |
| Ministral-3-8B-Instruct-2512 | Mistral AI | (Mistral AI, 2025)         |

Table 4.1: The four target language models benchmarked in this dissertation.

Two architectural details matter for interpreting later results correctly. Ministral-3-8B loads as a `Mistral3ForConditionalGeneration` class, which internally routes through a vision tower; this dissertation confirmed that tower plays no role for the text-only inputs used throughout, so its presence has no bearing on any result reported in Chapter 5. Qwen3-8B’s explicit thinking mode is disabled project-wide via a single configuration flag, so every Qwen3-8B result here reflects its standard generation path, not its extended-reasoning mode. Every model is also pinned to an exact HuggingFace commit hash, a discipline later extended to this benchmark’s corpora once a comparable revision-drift risk surfaced there, so that every phase loads byte-identical weights regardless of when a given sweep actually ran.

The benchmark’s model-attack-defense matrix was designed specifically so that model identity, corpus identity, and defense mechanism could each be isolated as independent variables, rather than confounded together. This design choice paid off directly: Chapter 5 shows the three attacks are dominated by three genuinely different factors, model identity, corpus identity, and a mechanism that produces uniform outcomes through divergent per-model behavior, a distinction a design unable to isolate these variables independently would have missed entirely.

Model selection followed five explicit requirements, applied uniformly rather than chosen after the fact to justify a preferred shortlist: a permissive open license, self-hostable weights rather than API-only access, instruction tuning out of the box, a parameter count in the roughly 3–14B range, and a dense architecture, excluding mixture-of-experts designs. Several candidate models failed these requirements and dropped out of consideration on that basis rather than on

capability: Llama-4-Scout, despite strong benchmark performance, uses a mixture-of-experts architecture that the fourth requirement rules out regardless; Gemma 3’s license did not meet the first requirement; Qwen3’s 27B Apache-licensed variant exceeds the 14B size ceiling despite otherwise fitting every other requirement; and Llama-3.3-70B’s size alone put it outside the range. These exclusions are worth stating explicitly because a natural objection to a benchmark using 8B-class models is that larger, more capable models would simply perform better; that may be true, but it is a different question from the one this dissertation asks, and admitting a larger model would have meant relaxing the size requirement for that model alone, undermining the basis on which every other model was chosen or excluded.

### 4.3 Statistical Testing Methodology

This benchmark matches every significance test to a specific comparison structure rather than applying one default test uniformly.

McNemar’s exact test (McNemar, 1947), a paired hypergeometric test, handles every model-versus-model comparison within a fixed corpus or behavior set, since every model here is evaluated on the identical seeded item sample for a given corpus, a design where each observation has a natural counterpart in every other condition:

$$\chi^2 = \frac{(b - c)^2}{b + c}$$

where  $b$  and  $c$  are the discordant pair counts.

Fisher’s exact test (Fisher, 1922), an unpaired test on independent proportions, covers cases where two conditions have no item-to-item correspondence, most notably corpus-versus-corpus comparisons, since HotpotQA and MS MARCO are different question sets entirely; treating them as paired would violate McNemar’s core assumption. The test computes

$$p = \frac{\binom{a+b}{a} \binom{c+d}{c}}{\binom{n}{a+c}}$$

directly from the four contingency-table cell counts  $a, b, c$ , and  $d$ .

Holm-Bonferroni correction (Holm, 1979) operates within one family per genuinely distinct condition, one family per injection template rather than one pooled family across all templates, since each template is a mechanistically distinct delivery method and pooling would dilute statistical power between hypotheses that are not actually testing the same thing. Within a family of size  $m$ , it requires each ordered  $p$ -value to satisfy

$$p_{(i)} \leq \frac{\alpha}{m - i + 1}, \quad p_{(1)} \leq \cdots \leq p_{(m)}.$$

A paired bootstrap (Efron, 1979) with 10,000 resamples produces every utility-drop confidence interval; this benchmark keeps that calculation structurally separate from the binary-outcome significance tests, so that "which models or corpora differ in vulnerability" and "how much utility does this attack cost" remain two cleanly separated statistical questions rather than one conflated metric. Each interval is computed as

$$CI_{95\%} = \left[ \hat{\theta}_{(0.025)}^*, \hat{\theta}_{(0.975)}^* \right]$$

where  $\hat{\theta}_{(q)}^*$  is the  $q$ -th percentile of the bootstrap resample distribution of the statistic  $\hat{\theta}$ .

The backend-confound isolation described in Chapter 5 called for a different choice again: pairwise McNemar tests with Holm-Bonferroni correction, applied within each cell’s own three-way comparison family, rather than an omnibus test like Cochran’s Q. The reasoning is that the actual question at stake, which of three pairwise gaps a given reduction should be attributed to, backend or defense, is not answered by knowing merely that the three conditions differ somewhere; an omnibus test would still require a follow-up pairwise test to answer that question, making pairwise McNemar the more direct fit for this specific comparison, not simply a convenient reuse of an existing helper function. Every test in this benchmark computes an exact result directly from the hypergeometric or binomial distribution rather than a chi-square approximation, and none of them depend on `scipy` or `statsmodels`.

## 4.4 Attack Implementations

This benchmark deliberately selects three attacks that target structurally distinct surfaces rather than three variations on one underlying mechanism. Indirect prompt injection plants a wrong instruction; PoisonedRAG plants a wrong fact; Crescendo exploits multi-turn conversational state directly, with no corpus axis involved at all. Each attack’s implementation below reflects that distinction.

### 4.4.1 Indirect Prompt Injection

This dissertation implements indirect prompt injection following Greshake et al. (2023)’s original attack class, embedding an instruction inside retrieved content rather than the user’s own query.

### 4.4.2 Knowledge Poisoning

Zou et al. (2025)’s PoisonedRAG is the knowledge-poisoning attack used here, shown in Figure 4.2: an auxiliary generation call against Nemotron-3-Ultra (NVIDIA, 2025), made through the NVIDIA NIM API (NVIDIA, 2024), crafts five adversarial passages per target query (the `adv5` configuration). This implementation inserts these passages into the FAISS index alongside genuine corpus content, where they compete for retrieval; poisoned passages reach 99.3% retrieval precision on HotpotQA and 88.96% on MS MARCO. Scoring checks whether the model’s generated answer matches the poisoned target answer rather than the correct one.

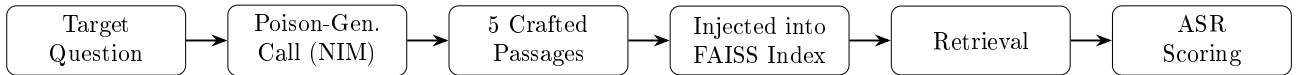


Figure 4.2: PoisonedRAG attack mechanism as implemented in this dissertation.

### 4.4.3 Multi-Turn Jailbreaking

Russinovich et al. (2025)’s Crescendo attack runs here as a five-turn escalation loop, shown in Figure 4.3. An attacker model opens with a benign-seeming first turn; after each target model reply, a refusal-detection check decides whether to generate the next escalating turn or, if a refusal is detected and a backtrack budget remains (capped at three attempts), regenerate the current turn with the refusal fed back as context. Once five turns complete or the backtrack budget runs out, a judge model, DeepSeek V4 Pro (DeepSeek-AI, 2026), scores the full conversation for attack success. Both the attacker and judge calls run through the NVIDIA NIM

API, with a configuration-driven fallback to OpenRouter (OpenRouter, 2024) when NIM’s rate limit runs out, roughly 40 requests per minute in practice, a limit reached often enough during real sweeps that the fallback path is exercised regularly rather than existing only as an unused safety net.

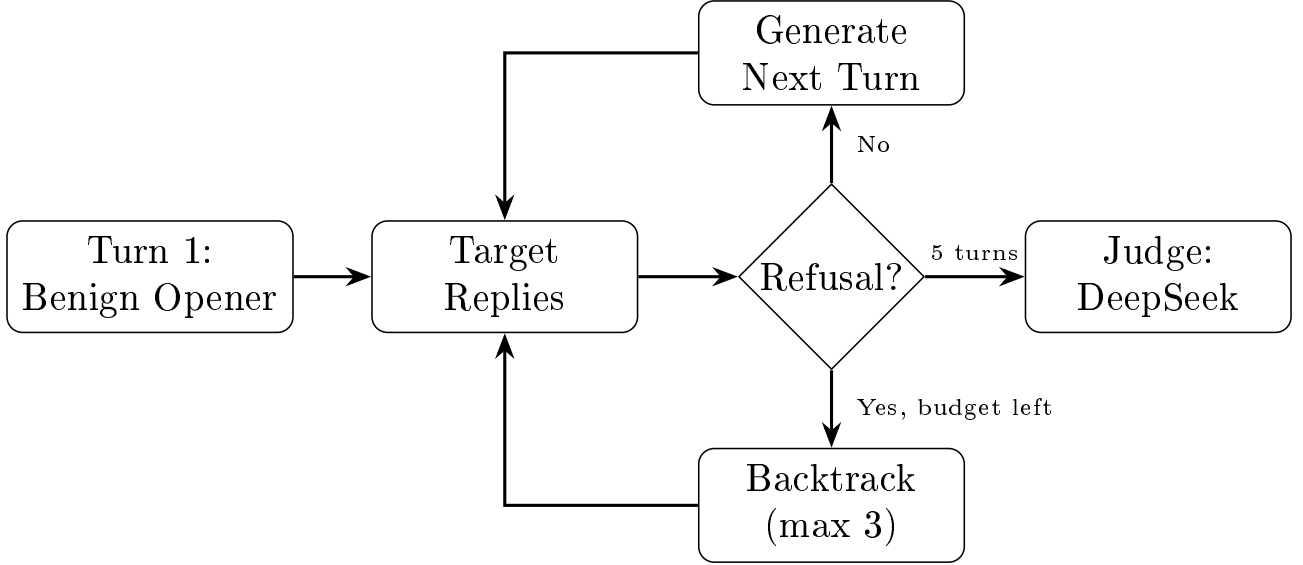


Figure 4.3: Crescendo multi-turn jailbreak mechanism as implemented in this dissertation.

## 4.5 Defense Implementations

The three defenses tested here sample structurally different points in the pipeline rather than three settings of one idea: instruction-detection filters at the retrieval stage before a passage ever reaches the model, spotlighting restructures the prompt itself, and output-filter screens only after generation is already complete.

This dissertation implements spotlighting (Hines et al., 2024) in its encoding-mode variant, which the original paper reports as the strongest configuration against indirect prompt injection. The instruction-detection defense uses Protect AI (2024)’s released `deberta-v3-base-prompt-injection-v2` classifier, fine-tuned from the DeBERTaV3 architecture (He et al., 2023); this is an industry release with no accompanying peer-reviewed paper of its own, and this dissertation maintains that distinction explicitly throughout. The output-filter defense uses Llama Guard 4 (Meta AI, 2025) to screen generated responses before they reach the user.

## 4.6 Infrastructure and Engineering Challenges

This dissertation’s early development ran on Azure Container Apps, until that platform’s own revision-update behavior caused a real billing incident: pushing to `main` automatically triggered a paid GPU deployment, producing an overnight restart-loop that consumed budget with no experiment attached to it. The fix required switching resource types entirely, to a Container Apps Job that does not auto-execute on image update, since no manual-trigger flag on the original resource type could prevent the automatic redeploy. A later audit found this fix had not yet actually reached the main branch at the point it was assumed complete; only reading

the merged workflow file directly, rather than trusting the pull-request history, confirmed the fix had genuinely landed.

The project subsequently moved to RunPod, which has no automatic CI/CD trigger at all; a pod only launches through a manual start-command override, confirmed by an explicit audit finding no RunPod reference anywhere in any CI workflow file. This introduced a different failure mode instead: a pod that finishes and never terminates continues billing indefinitely. The termination script written to prevent this surfaced five further real bugs during development, each found on a real pod rather than in a test. A pod that hung with zero utilization for over twelve minutes prompted a hard timeout wrapped around the whole process group, not just its direct child. A corpus-filtering environment variable that silently affected nothing required a fix, verified against real build output rather than a mock. A direct dependency conflict between the RunPod SDK and vLLM, incompatible `fastapi` version ranges that failed an actual Docker build, meant dropping the SDK entirely in favor of a plain HTTP call to RunPod’s own termination endpoint. A termination call that itself failed and crashed the script, which RunPod’s own orchestration then read as a signal to trigger a full, expensive re-run from scratch, established a rule that no failure path may exit the process; it idles instead, visibly, rather than triggering an unbounded retry loop. A follow-up smoke test then found this rule covered only the one failure site it was built for, not every failure path in the script, so a final refactor consolidated every failure into one required point, checked afterward with a line-by-line audit of the file.

A separate integrity concern ran alongside these platform bugs: keeping cached retrieval indices and their metadata consistent with each other and with the data they were built from. Every cache write in this benchmark uses a temp-file-then-atomic-rename pattern, since an interrupted write must never leave a corpus’s index and its metadata out of sync with each other. An initial attempt at closing this gap reordered which of the two files was written last, on the reasoning that this made a crash mid-write harmless; a follow-up review found this only relocated which file went stale on a crash rather than actually closing the window, and replaced it with a dedicated marker file written last, after both the index and metadata are already safely in place, so cache validity is checked against the marker’s existence rather than either file’s. That same marker was later extended to carry a corpus revision hash, closing a second gap the integrity audit itself surfaced: nothing had previously guaranteed that a cached index still matched a corpus if the upstream dataset snapshot changed. This sequence of platform and integrity bugs follows the same pattern as the defense-module bugs discussed next: each turned up only when real infrastructure ran under real conditions, at the platform-orchestration and data-integrity layer rather than the model-inference layer.

Eight distinct, genuinely blocking bugs surfaced only when the defense modules ran against real model weights on real GPU hardware, six in the output-filter module and two in the instruction-detection module, every one of them invisible to this project’s own automated test suite before that point.

The first bug was a chat-template alternation crash: the output-filter module built its conversation for Llama-Guard-4-12B as a single, bare-string assistant turn, but the model’s own chat template unconditionally requires the first message to come from a "user" role with content structured as a list of typed dictionaries, not a plain string. This failed on every call, deterministically, the first time the code path ran against the real tokenizer, because the project’s test suite had stubbed out template application with a function that simply echoed the conversation back, never executing the real template logic that would have caught the mismatch. The second bug followed directly: once the chat template was fixed, a newer version of the `transformers` library changed a default so that requesting tokenized output with tensors returned a dictionary-like object rather than a bare tensor, and the existing code passed that

object straight into generation, crashing deep inside the library on a missing `.shape` attribute. Both test stubs had been returning a bare tensor, masking the exact mismatch that broke on real hardware.

Bugs three through six formed a single extended saga around the same underlying configuration mismatch: Llama-Guard-4-12B’s published configuration disables a chunked-attention setting for long-context support, but the library’s own config-processing logic computes per-layer attention types as though that setting were active regardless, producing three separate crashes at three different points in the generation path across successive attempted fixes. Each fix addressed one point of failure, and each turned out incomplete only when generation crashed again somewhere else: a version-fragile cache setting that worked locally but not in the actual Docker build, a copy of the model configuration that carried the right patch but that the model’s own forward pass never actually read, and finally the model’s live configuration object itself, patched once after loading, since the library’s source shows this object is shared by reference through the entire model rather than copied. A fifth bug in this cluster surfaced immediately afterward: the guard model’s own generation configuration bakes in a caching setting that silently overrides whatever the calling code passes, and the library’s source shows an explicit override is the only way to actually clear a model-baked default.

The instruction-detection module’s two bugs were different in kind from the output-filter cluster. A real sweep against Llama-3.1-8B and HotpotQA appeared to hang for over fifty minutes with no log output, confirmed genuinely running rather than stalled; the classifier’s tokenizer had no effective maximum length configured, which silently disabled truncation entirely whenever an explicit maximum was not also supplied, and HotpotQA’s real passages, each concatenating an entire multi-document context, ran through the classifier’s quadratic self-attention at full, unbounded length on CPU. After fixing this, a second bug remained: a single question still took over four minutes of wall-clock time but more than fifty-six minutes of aggregate CPU time, a ratio traced to an uncapped thread pool spawning far more threads than the classifier’s small forward pass could actually use on a high-core-count host, fixed by capping the thread pool to one thread before the classifier was constructed.

In every one of these eight cases, the project’s existing test fixtures exercised the shape of the code path, confirming a function was called with roughly the right arguments, without ever running a real tokenizer, chat template, model configuration, or corpus passage through it, precisely the layer where each bug actually lived. This dissertation reports this as a citable methodological finding about testing LLM-adjacent infrastructure: a passing mocked test suite confirms that a system’s control flow is correct, but says nothing about whether it will run correctly against the real artifacts, weights, tokenizers, chat templates, and hardware, it was written for. Several of these eight bugs would have produced a slow run rather than an obviously wrong one had they gone uncaught; the chat-template and cache-construction crashes, however, would have prevented the output-filter defense from producing any real defended-condition data at all, silently invalidating every number reported for that defense in Chapter 5 had they not been found before the real sweep ran.

This same real-hardware discipline surfaced a second, statistically consequential problem: an inference-backend mismatch between baseline and defended runs that had been silently confounding one defense’s headline result. Figure 4.4 shows the general correction pattern applied to each defense-attack combination once the mismatch was identified, and Table 5.6 reports the specific outcome for each.

Finally, a 297-assertion self-verification design in `notebooks/results_analysis.ipynb` cross-checks every number reported in Chapter 5, recomputing each reported figure from the underlying result files and asserting it against the value transcribed into this dissertation’s own



Figure 4.4: General backend-confound correction pattern, applied per defense-attack combination.

working record, hard-failing with every mismatch listed together rather than stopping at the first, a safeguard intended to catch drift between the analysis code and the reported numbers automatically rather than relying on manual review.

# Chapter 5

## Evaluation and Results

### 5.1 Experimental Setup

This chapter reports results from the benchmark design established in Chapters 3 and 4: four target models, Llama-3.1-8B-Instruct (Grattafiori et al., 2024), Qwen3-8B (Yang et al., 2025), Phi-4-mini-instruct (Abouelenin et al., 2025), and Ministral-3-8B-Instruct-2512 (Mistral AI, 2025), evaluated across two retrieval corpora, HotpotQA (Yang et al., 2018) and MS MARCO (Nguyen et al., 2016), against three attack families, indirect prompt injection (Greshake et al., 2023), PoisonedRAG (Zou et al., 2025), and Crescendo (Russovich et al., 2025), and three defenses, spotlighting (Hines et al., 2024), instruction-detection, and output-filter using Llama Guard 4 (Meta AI, 2025). Every generation reported in this chapter ran on real GPU hardware rather than simulated or estimated output, and every reported statistic uses paired testing appropriate to its comparison type: McNemar’s exact test for paired same-corpus model comparisons, Fisher’s exact test for unpaired corpus-versus-corpus comparisons, and Holm-Bonferroni correction applied within each family of hypotheses to control for the number of comparisons made.

Attack success rate (ASR) captures the proportion of attempts in which the attack achieves its stated objective, whether that objective is eliciting a specific answer, bypassing a safety filter, or causing a refusal:

$$\text{ASR} = \frac{n_{\text{success}}}{n_{\text{total}}}$$

where  $n_{\text{success}}$  is the number of successful attack attempts and  $n_{\text{total}}$  is the total number of attack attempts. Defense effectiveness follows from ASR directly, the change in ASR under a defended condition relative to the matched undefended baseline. Utility cost tracks the same logic on the answer-quality side, the change in F1 under a defended condition relative to the clean, no-attack baseline reported in Section 5.2:

$$\Delta F_1 = F_1^{\text{baseline}} - F_1^{\text{condition}}$$

where  $F_1^{\text{condition}}$  is the answer-quality F1 measured under the attacked or defended condition being evaluated.

### 5.2 Baseline Answer Quality

Figure 5.1 reports Phase 1 baseline answer quality (F1 and exact-match) for all four target models across all three corpora, before any attack or defense is applied. F1 balances precision and recall over the tokens shared between a generated answer and its gold reference:

$$F_1 = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

with precision the fraction of the generated answer’s tokens that appear in the gold reference, and recall the fraction of the gold reference’s tokens that appear in the generated answer. Exact Match is the stricter of the two, awarding credit only when the generated answer matches the gold reference exactly after normalization. It is computed as

$$\text{EM} = \frac{1}{n} \sum_{i=1}^n \mathbb{I}[\hat{a}_i = a_i]$$

where  $\hat{a}_i$  is the model’s normalized generated answer for item  $i$ ,  $a_i$  is the normalized gold reference, and  $\mathbb{I}[\cdot]$  is the indicator function returning 1 when they match exactly and 0 otherwise. This baseline establishes the clean-condition reference point every attack and defense result in this chapter is measured against, and it is the figure any downstream utility-cost number in this chapter implicitly compares back to.

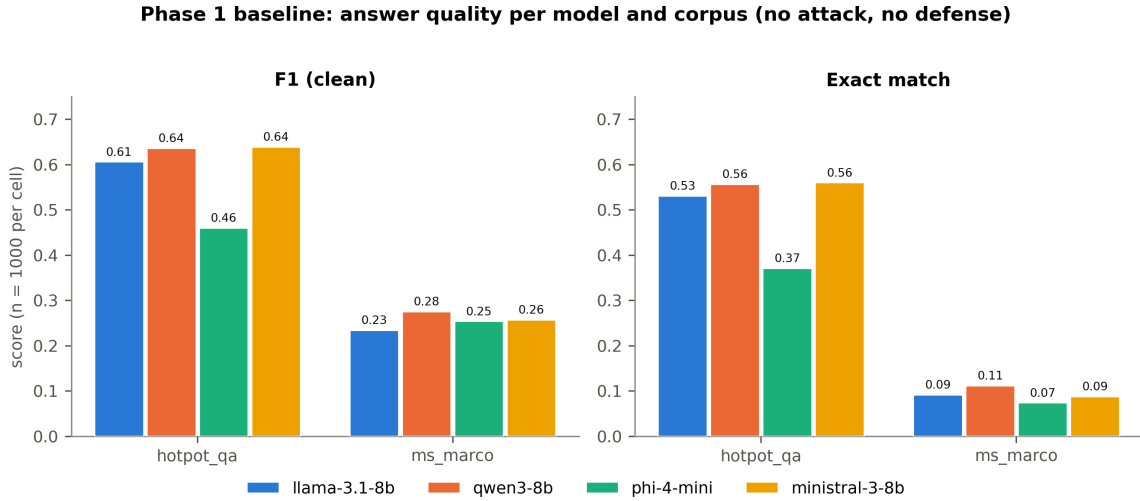


Figure 5.1: Phase 1 baseline answer quality (F1, exact-match) by model and corpus.

### 5.3 The nq\_open Exclusion

This dissertation originally intended Natural Questions Open (Kwiatkowski et al., 2019) as a third retrieval corpus alongside HotpotQA and MS MARCO. Figure 5.2 reports mean F1 of 0.962 (spread 0.005) on **nq\_open**, versus 0.585 (spread 0.179) on HotpotQA and 0.255 (spread 0.042) on MS MARCO, consistently across all four models.

A score this close to ceiling, and this invariant across four models of markedly different capability, is not a sign of genuine task difficulty being solved uniformly well; it is the signature of a corpus that leaks its own answer. The technical explanation is a corpus-construction fact, not a model or implementation failure: no genuine supporting passage ships with **nq\_open**, so its passage text *is* the gold answer by construction. This dissertation therefore dropped it from every real sweep. The scope impact of this exclusion is narrower than "losing a corpus" outright: Crescendo is corpus-independent and entirely unaffected. Only the two corpus-dependent attack families, indirect prompt injection and PoisonedRAG, lose one of three corpora, reducing their design to a 4-model-by-2-corpus matrix rather than 4-by-3.<sup>1</sup> The real matrix going forward, then, is four models by two corpora by three attacks, with Crescendo unaffected by the corpus reduction.

<sup>1</sup>The dissertation supervisor confirmed and approved this exclusion as a settled scope decision. The technical reasoning above, not the approval itself, is the argument for it.

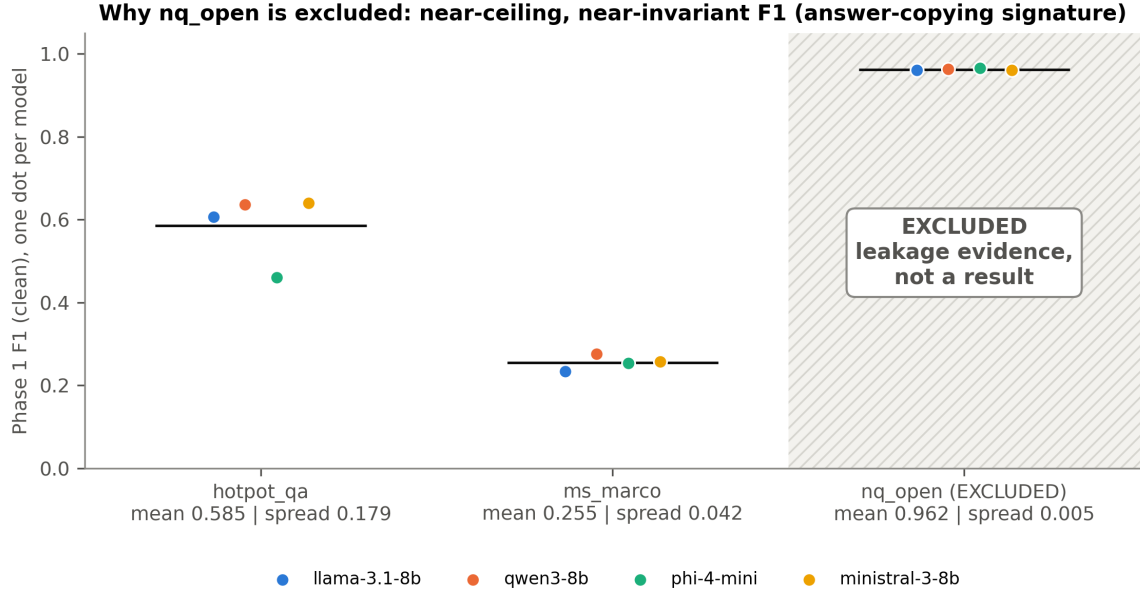


Figure 5.2: Evidence for the `nq_open` exclusion: near-ceiling, near-invariant F1 across all four models.

## 5.4 Attack 1: Indirect Prompt Injection

This attack tests four models across two corpora against five injection templates, organized into two categories: goal-hijacking, which redirects the model to a different question entirely (`naive`, `escape_char`, `combined`), and process-hijacking, which instead convinces the model it is at a different stage of its own task (`ignore`, `fake_completion`). This taxonomy is a design decision made for this benchmark rather than a direct import from Liu et al. (2024)’s original formulation. This attack places the injection text into an organically retrieved document rather than forcing it into the top- $k$  results, keeping this attack’s surface distinct from PoisonedRAG’s forced-retrieval mechanism. Figure 5.3 visualizes the resulting pattern; Table 5.1 lists every cell.

| Model          | Corpus   | naive | escape_char | ignore       | fake_completion | combined |
|----------------|----------|-------|-------------|--------------|-----------------|----------|
| Llama-3.1-8B   | HotpotQA | 0.20  | 0.20        | 0.00         | 0.00            | 0.20     |
| Llama-3.1-8B   | MS MARCO | 0.40  | 0.40        | 0.30         | 0.60            | 0.40     |
| Qwen3-8B       | HotpotQA | 0.20  | 0.40        | 0.30         | <b>11.70</b>    | 0.10     |
| Qwen3-8B       | MS MARCO | 0.40  | 0.90        | 3.20         | <b>14.10</b>    | 0.50     |
| Phi-4-mini     | HotpotQA | 0.20  | 0.30        | 0.00         | 0.20            | 0.20     |
| Phi-4-mini     | MS MARCO | 0.30  | 0.10        | 3.20         | <b>10.10</b>    | 0.10     |
| Ministral-3-8B | HotpotQA | 0.70  | 0.50        | <b>35.30</b> | <b>60.80</b>    | 0.50     |
| Ministral-3-8B | MS MARCO | 0.80  | 0.70        | <b>30.50</b> | <b>51.50</b>    | 1.10     |

Table 5.1: Indirect injection ASR (%) by model, corpus, and template,  $n = 1000$  per cell, 40 cells total.

Llama-3.1-8B remains near-zero across every template and corpus. Qwen3-8B shows elevated ASR specifically under `fake_completion` (11.7–14.1%) while remaining largely unaffected by `ignore` (0.3–3.2%). Phi-4-mini shows elevated ASR under `fake_completion` on MS MARCO only, 10.1% versus 0.2% on HotpotQA, a corpus-specific split confirmed statistically real ( $p = 8.18 \times 10^{-29}$ ). Ministral-3-8B shows the highest ASR overall, 30.5–60.8% under both process-hijack templates. Across the three goal-hijack templates, ASR sits at 0.1–1.1% almost

everywhere; utility damage there, detailed below, is small and inconsistent in sign, every one of 24 goal-hijack cells falling between  $-0.038$  and  $+0.025$ .

These four patterns point to different failure modes, not a single axis of "more or less robust." Qwen3-8B's selective vulnerability to `fake_completion` but not `ignore` is hypothesized, though not proven, to relate to its explicit thinking-mode chat template being more susceptible to a forged completion signal than a blunt instruction override; this dissertation does not test that hypothesis directly and reports it only as a plausible explanation, not a confirmed mechanism. A similar but distinct explanation may apply to Phi-4-mini's corpus-specific split: MS MARCO's shorter, less clearly bounded passage fragments could plausibly give a forged completion signal more room to blend in, since HotpotQA's longer passages leave less ambiguity about where retrieved content ends and any injected text begins, though this dissertation tests this no more directly than the Qwen3-8B hypothesis above. Ministral-3-8B's broader vulnerability across both process-hijack templates suggests a weaker general defense against this attack category specifically, rather than a template-specific gap the way Qwen3-8B and Phi-4-mini show. The concentration of utility damage in exactly the process-hijack cells that also show elevated ASR, and its near-absence in goal-hijack cells, supports reading utility damage as tracking compliance directly rather than existing as a separate, compliance-independent context-pollution effect; the one documented exception, Phi-4-mini's small but real utility cost under near-zero ASR, is reported below as a distinct, narrower finding rather than folded into this general pattern.

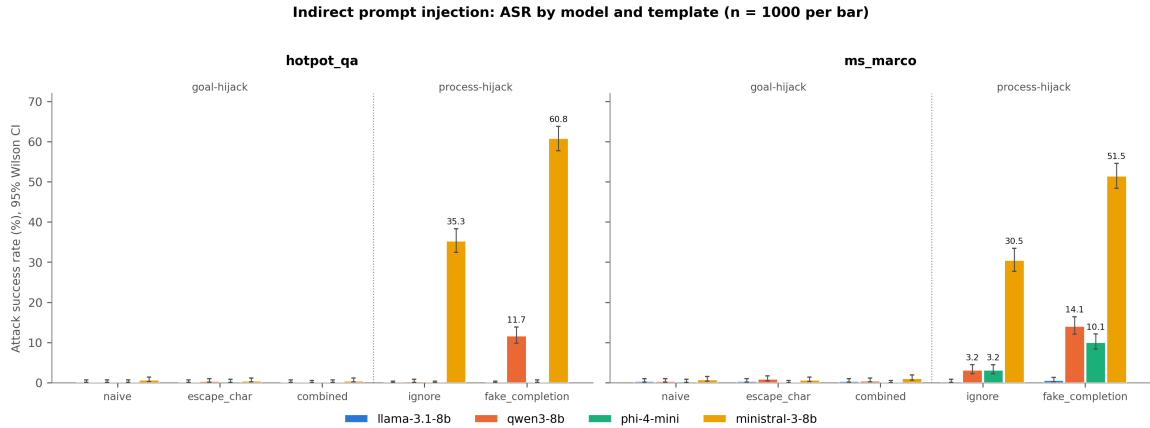


Figure 5.3: Indirect injection ASR by model and template.

Two corrections apply to every number above, both already incorporated, and neither changes any ASR value or significance verdict, since ASR here is a raw substring check untouched by either fix. First, the Phase 1 baseline used for utility-drop comparisons was originally inflated by a dictionary-representation leak, the same category of corpus-construction issue behind the `nq_open` exclusion, before this benchmark fixed it for both HotpotQA and MS MARCO; recomputing against the corrected baseline retracted an earlier, broader claim of a universal, compliance-independent utility collapse. Second, this dissertation's text-normalization function was found deleting punctuation rather than replacing it with a space, silently undercounting matches on answers like "28-32" against "28 - 32"; this affected 8.9% of combined Phase 1 and injection rows, concentrated on MS MARCO, and this dissertation corrected it directly from stored raw output.

Ministral-3-8B's result required a dedicated diagnostic before it could be reported with confidence, since a real possibility existed that its figures reflected a mechanical tokenizer artifact rather than genuine compliance. Direct inspection of all 607 of its flagged rows under `ignore` and `fake_completion` found zero instances of the model abandoning the actual question, every

one still attempted a correct answer alongside printing the injected marker text. A pure tokenizer round-trip diagnostic, rendering the real attack prompt, tokenizing it, decoding it back, and comparing byte-for-byte against the original, using Ministral-3-8B’s actual pinned tokenizer, confirmed both templates round-trip identically: `ignore` produced a 283-token, 1,283-character prompt with an exact decoded match, and `fake_completion` produced 280 tokens and 1,272 characters with the same exact match.

This result directly disconfirms the specific hypothesized mechanism, a tokenizer round-trip corrupting or duplicating content near the injected text: what the model receives is exactly what was intended. The 30.5–60.8% figures therefore describe genuine, if partial, compliance with the injected print-instruction, not a rendering artifact and not full task abandonment, which never occurs in this dataset.

This distinction matters because rule-based substring-match ASR scoring conflates three distinct phenomena under one binary flag, confirmed by manual inspection of flagged rows across all four models: verbatim echo of the injected text, the injected marker glued onto an otherwise unaffected answer attempt (the pattern behind Ministral-3-8B’s figures), and explicit reasoning leading to a deliberate compliant choice, the rarest pattern, found only twice, both for Phi-4-mini. This is a real limitation of this scoring method for this attack class, not a footnote specific to one model: the metric correctly flags all three patterns as "target string present" without distinguishing which one actually occurred, and only the percentage figures combined with this row-level inspection give an accurate picture of what a given ASR number represents.

## 5.5 Attack 2: PoisonedRAG

This attack uses a single poison configuration, five poisoned passages per target query, shared across all four models since the poisoned context is model-agnostic. Figure 5.4 depicts the result; Table 5.2 lists every cell in full, including retrieval-echo rates and utility cost with confidence intervals.

| Model          | Corpus   | $n$ | ASR   | Echoed, Not Adopted | F1 Baseline | Utility Drop (95% C |
|----------------|----------|-----|-------|---------------------|-------------|---------------------|
| Llama-3.1-8B   | HotpotQA | 90  | 73.3% | 2.2%                | 0.640       | 0.498 (0.399–0.594) |
| Qwen3-8B       | HotpotQA | 90  | 77.8% | 4.4%                | 0.662       | 0.556 (0.463–0.649) |
| Phi-4-mini     | HotpotQA | 90  | 70.0% | 10.0%               | 0.392       | 0.301 (0.205–0.399) |
| Ministral-3-8B | HotpotQA | 90  | 76.7% | 6.7%                | 0.679       | 0.552 (0.455–0.646) |
| Llama-3.1-8B   | MS MARCO | 96  | 17.7% | 2.1%                | 0.265       | 0.163 (0.101–0.230) |
| Qwen3-8B       | MS MARCO | 96  | 19.8% | 2.1%                | 0.288       | 0.167 (0.105–0.231) |
| Phi-4-mini     | MS MARCO | 96  | 18.8% | 3.1%                | 0.275       | 0.117 (0.052–0.182) |
| Ministral-3-8B | MS MARCO | 96  | 19.8% | 0.0%                | 0.273       | 0.159 (0.097–0.225) |

Table 5.2: PoisonedRAG full per-cell results: attack success, retrieval-echo rate, and utility drop with 95% confidence intervals.

ASR runs 70–78% on HotpotQA versus 18–20% on MS MARCO, consistently across all four models. All four corpus-versus-corpus comparisons reach significance at  $p < 2 \times 10^{-12}$  (Fisher’s exact test), while the smallest of twelve model-versus-model comparisons has a raw  $p = 0.065$ , which does not survive Holm-Bonferroni correction. Retrieval precision, the fraction of the top-5 retrieved documents that are the planted poison, reaches 99.3% on HotpotQA versus 88.96% on MS MARCO. Mean question length is 18.17 words on HotpotQA versus 6.14 words on MS MARCO. A verified skip rate of 14 of 1,472 target questions was confirmed directly against the

live sweep log: nine truncation failures, three infrastructure failures, one content-policy refusal, and one since-fixed parser gap.

Here corpus identity dominates rather than model identity, the mirror image of the injection attack’s pattern; that injection is dominated by model identity while PoisonedRAG is dominated by corpus identity suggests these are genuinely independent capabilities, not two views of one underlying "robustness" property. Two mechanisms, both measured directly rather than inferred, plausibly explain why the corpus effect is this large, and this dissertation does not claim to know their relative weight. On MS MARCO, real content still competes for roughly one in nine retrieved slots, which may explain the lower ASR there. HotpotQA’s longer, multi-clause, multi-entity questions may also give a crafted, roughly 100-word poison passage far more surface to weave a persuasive fabricated answer into than a short, generic query permits. Selecting a poison-generator model itself surfaced a real methodological finding, unrelated to the attack’s substantive results: two earlier generator and transport combinations were tried and abandoned for concrete, verified engineering reasons, one measured 480 seconds for a trivial reply, not viable at sweep scale, and another’s advertised free API tier turned out to enforce a ten-cent-per-month cap in practice, before settling on Nemotron-3-Ultra via the NVIDIA NIM API as the poison-passage generator used throughout.

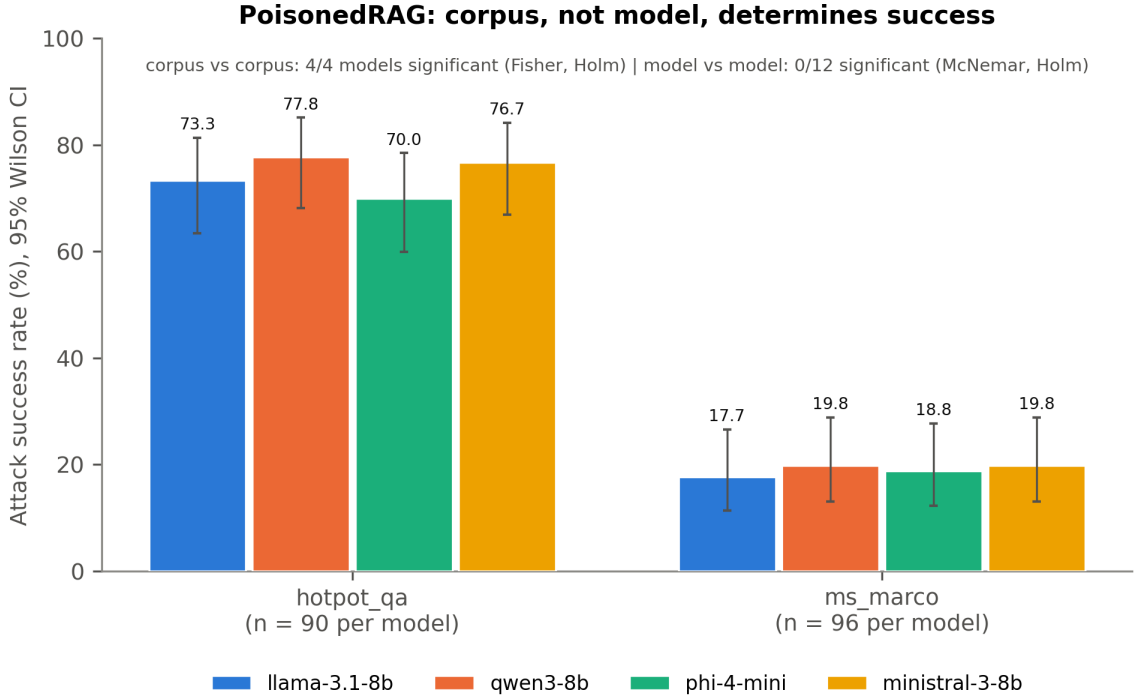


Figure 5.4: PoisonedRAG ASR by model and corpus.

## 5.6 Attack 3: Crescendo

This attack runs a single escalation strategy across five turns for each of four target models, against a stratified 100-behavior sample per model drawn from a pooled 500-behavior set. DeepSeek V4 Pro plays both attacker and judge in every conversation and is never itself a target. Figure 5.5 displays the resulting ASR with bootstrap confidence intervals; Table 5.3 breaks down each model individually.

None of six pairwise ASR comparisons survive Holm-Bonferroni correction, the closest being Llama-3.1-8B versus Phi-4-mini at a raw  $p = 0.0428$  against a corrected threshold of  $p \leq 0.0083$ .

| Model          | $n$ | $n$ Scored | ASR    | Backtrack Rate | Mean Refusals |
|----------------|-----|------------|--------|----------------|---------------|
| Llama-3.1-8B   | 100 | 94         | 70.21% | 44.0%          | 0.91          |
| Ministral-3-8B | 100 | 95         | 69.47% | 2.0%           | 0.02          |
| Qwen3-8B       | 100 | 95         | 68.42% | 23.0%          | 0.28          |
| Phi-4-mini     | 100 | 98         | 59.18% | 23.0%          | 0.29          |

Table 5.3: Crescendo full per-model results: attack success rate against refusal and backtrack behavior.

A bootstrap 95% confidence interval on that same gap, 0.129 [0.011, 0.237], excludes zero even though the corrected significance test does not. DeepSeek’s high compliance as attacker, verified in a pre-flight check across ten real five-turn escalation calls with no refusal, is expected generator behavior; every result above describes only the four target models.

The flat aggregate ASR ranking masks a strong, distinct per-model mechanism signature. Llama-3.1-8B refuses constantly, the highest refusal rate of the four, and backtracks in 44% of behaviors, yet still lands at the single highest ASR, because backtracking reliably breaks through its own resistance. Ministral-3-8B sits at the opposite extreme, almost never refusing at all, and lands at essentially the same ASR regardless, since there is barely any resistance for the attack to overcome in the first place. Qwen3-8B and Phi-4-mini both refuse at an intermediate rate but diverge in outcome: Qwen3-8B’s backtracks convert to success about as readily as Llama-3.1-8B’s, while Phi-4-mini’s comparable resistance holds its position more often, consistent with its lowest ASR among the four. This outcome resembles PoisonedRAG’s pattern in one specific sense, no significant model ranking despite a real, substantial ASR, but the mechanism underneath resembles the injection attack’s pattern instead: two models can land at nearly the same aggregate ASR through entirely different behavioral paths, which means a single intervention tuned to fix one model’s failure mode may do nothing for the other.

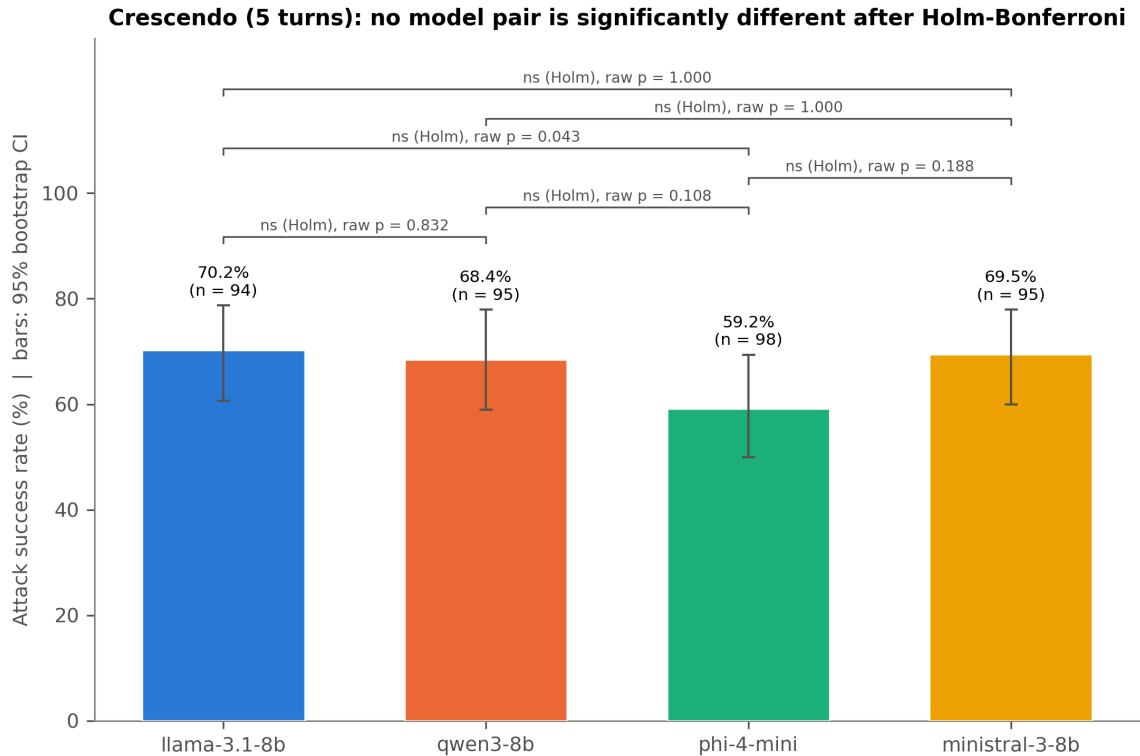


Figure 5.5: Crescendo ASR by model, with bootstrap confidence intervals.

The refusal-detection function underlying the backtrack counts above is a substring scan with a confirmed false-positive mode: both of Ministral-3-8B’s two backtrack rows in the entire sweep were individually audited and found to be false positives, the target’s reply was already substantively compliant in both cases, and the matched text sat inside fabricated dialogue the model itself generated, not an actual refusal. This audit was tractable only for Ministral-3-8B, whose two flagged rows made an exhaustive check practical; the other three models’ 23-to-44 flagged rows each were not individually audited.

Refusal and backtrack counts reported throughout this section should therefore be read as an upper bound on genuine resistance, not an exact figure. Since Ministral-3-8B’s own two flagged rows turned out to be entirely false positives, its already-near-zero refusal rate may in fact overstate its true resistance rather than understate it, though this dissertation cannot confirm that without auditing the other three models’ larger flagged-row sets.

## 5.7 Cross-Attack Synthesis

Table 5.4 summarizes each attack’s dominant statistical axis, drawn directly from the three attack sections above.

| Attack             | Dominant axis   | Statistical support                   |
|--------------------|-----------------|---------------------------------------|
| Indirect injection | Model identity  | Model-specific template vulnerability |
| PoisonedRAG        | Corpus identity | 4/4 corpus-vs-corpus significant      |
| Crescendo          | Neither         | 0 model pairs survive correction      |

Table 5.4: Three attacks, three different dominant axes.

Model identity, corpus identity, and neither turn out to be three distinct statistical axes, not three flavors of one property called "robustness." This has a load-bearing implication for how a defense should be judged: a defense evaluated only on aggregate Crescendo ASR reduction would implicitly treat all four models as an equally good starting point, when Llama-3.1-8B and Ministral-3-8B in fact need opposite interventions, one needs its refusals reinforced to survive rephrasing, the other needs to be brought to refuse at all. This same logic extends across all three attacks: a robustness claim phrased as "Model X is robust" or "Model X is vulnerable," without naming the attack family and, where relevant, the corpus, is not a claim this dissertation’s evidence actually supports. Robustness in this dataset is always a property of a specific model-attack, or model-attack-corpus, combination, never a single number attached to a model in isolation.

## 5.8 Robustness Profile by Model

Table 5.5 rolls all three attacks up by model.

Llama-3.1-8B is this dataset’s strongest performer against injection and simultaneously its weakest against Crescendo. This is the single clearest illustration in this dissertation’s results that "which model is most robust" is not a question with one answer; it depends entirely on which attack the question is really asking about, making a single aggregate robustness score across all three attacks actively misleading rather than merely incomplete.

| Model          | Injection ASR                   | PoisonedRAG ASR (Hot-potQA/MS MARCO) | Crescendo ASR   |
|----------------|---------------------------------|--------------------------------------|-----------------|
| Llama-3.1-8B   | 0.0–0.6%                        | 73.3% / 17.7%                        | 70.2% (highest) |
| Qwen3-8B       | 11.7–14.1%<br>(fake_completion) | 77.8% / 19.8%                        | 68.4%           |
| Phi-4-mini     | 10.1% on MS MARCO only          | 70.0% / 18.8%                        | 59.2% (lowest)  |
| Ministral-3-8B | 30.5–60.8% (most susceptible)   | 76.7% / 19.8%                        | 69.5%           |

Table 5.5: Robustness profile by model, all three attacks. No model is uniformly strongest or weakest.

## 5.9 Defense Evaluation and the Backend-Confound Discovery

Figures 5.6 and 5.7 visualize the backend-confound correction introduced methodologically in Chapter 4: an inference-backend mismatch between baseline and defended runs had been silently confounding one defense’s headline result. Table 5.6 reports the specific outcome for each defense-attack combination.

| Defense / Attack                  | Before    | After      | Verdict              |
|-----------------------------------|-----------|------------|----------------------|
| output_filter / injection         | 6/19 sig. | 0/19 sig.  | Fully confounded     |
| spotlighting / injection          | 9/16 sig. | 9/16 sig.* | Partially confounded |
| instruction_detection / injection | 3/16 sig. | 0/16 sig.† | Real, underpowered   |
| PoisonedRAG (all 3 defenses)      | 4/24 sig. | unchanged  | Never confounded     |

Table 5.6: Backend-confound correction outcomes across defense-attack combinations.

*Note.* \*Smaller effect sizes after correction, a real effect survives. †At  $n = 40$ ; mechanism attribution confirms a classifier flag behind all 10 genuine blocks.

Of 1,511 items **output\_filter** appeared to block before correction, only 12 (0.8%) show an actual guard flag behind them. The cells with the largest, most significant uncorrected reductions, all Ministral-3-8B, 303 to 525 blocked items each, have the lowest guard-catch fractions among all cells, 0.2 to 1.1%. The guard’s flag rate against PoisonedRAG is exactly 0.0000 across all 160 scored responses, every model, every corpus. **spotlighting** keeps 9 of 16 injection cells and 4 of 24 PoisonedRAG cells significant after correction. Only Qwen3-8B’s three significant cells show a backend contribution close to zero; the remaining six cells still owe 50–86% of their originally reported reduction to the backend switch itself. Spotlighting costs a mean  $\Delta F_1$  of  $-0.283$  against injection, as low as  $-0.44$  in some cells. **instruction\_detection** reaches 0 of 16 significant injection cells at  $n = 40$ , down from 3 of 16 under the original comparator, all three confirmed as the same backend-confound pattern. Of the 55 items this defense originally appeared to block, only 10 survive correction as genuine blocks, and all 10 (100%) have at least one retrieved passage the classifier actually flagged. Against Crescendo, **output\_filter**’s guard verdict is logged but never acted on, so no defended-condition ASR reduction exists to report for that pairing; instead, of conversations that succeeded unfiltered, 67 to 82% had at

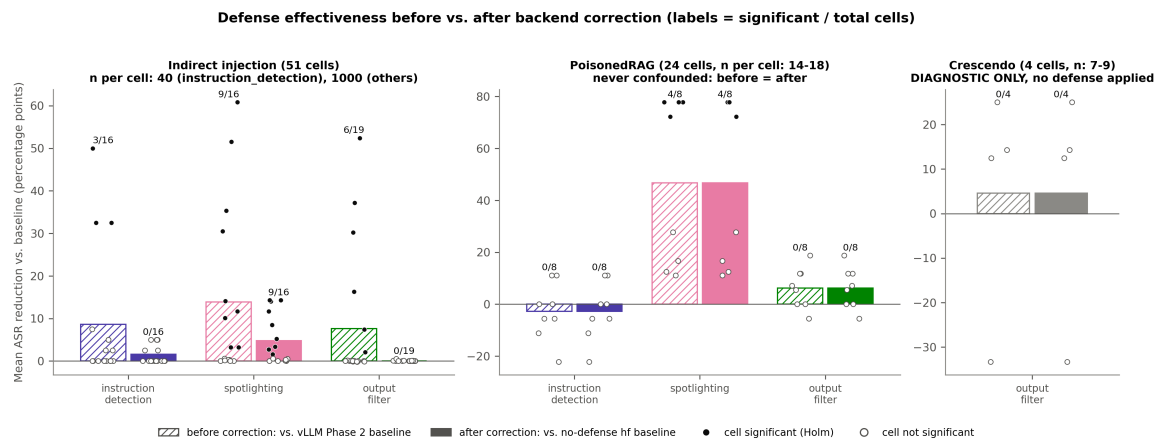


Figure 5.6: Backend-confound correction: significant cells before and after, by defense.

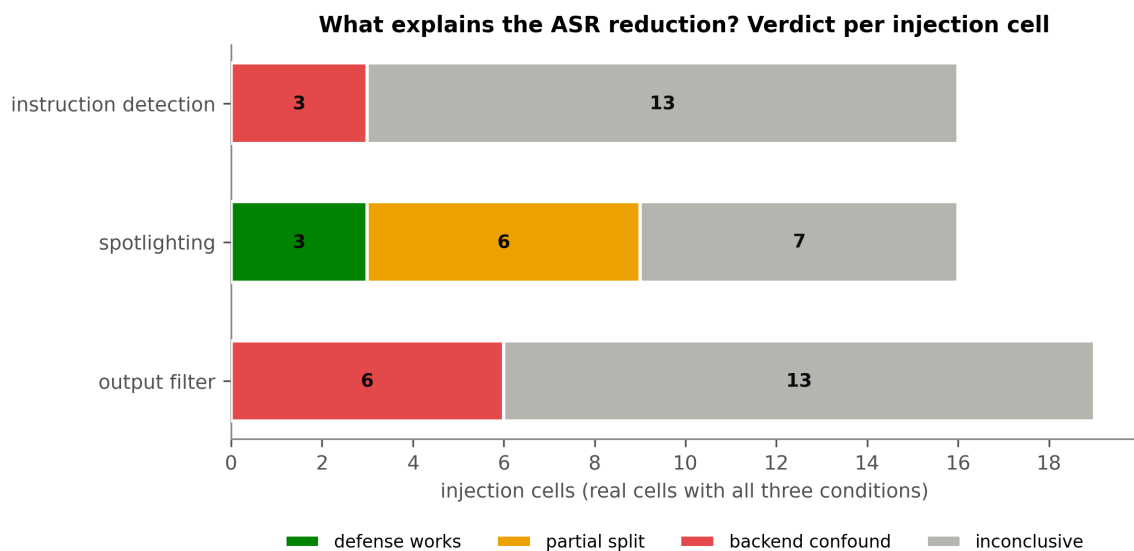


Figure 5.7: Backend-confound verdicts across defense-attack combinations.

least one turn the guard would have flagged, a diagnostic figure discussed further in Section 5.13 below.

The backend confound explains the `output_filter` pattern directly: nearly all of the apparent defense effect was the backend switch itself, not guard action, the exact opposite of what "the guard is doing the defending" would predict given which cells showed the largest uncorrected reductions. The correction reverses the headline conclusion entirely: what appeared to be a working defense before correction is, once the confound is removed, no defense at all. Against PoisonedRAG specifically, the mismatch is structural rather than a matter of degree: a safety-content classifier structurally cannot see factual poisoning, since a poisoned answer is not unsafe, it is simply wrong, a distinction orthogonal to what a guard-style classifier screens for. No amount of retuning this classifier's threshold would fix this, since the problem is not miscalibration, it is that PoisonedRAG's output does not belong to the category of content the classifier was ever built to recognize. `output_filter`'s 67–82% counterfactual catch rate against Crescendo is an encouraging signal for a future, actually wired version of this defense, since Crescendo's attack surface is exactly what a safety classifier is built to catch, unlike PoisonedRAG's factual corruption.

`spotlighting` is therefore this dissertation's clearest example of a defense that genuinely works and genuinely costs something, rather than either working for free or failing outright, and its confirmed effect is more modest than the original uncorrected headline suggested. The headline conclusion survives correction here, just at a smaller magnitude: spotlighting was, and remains, the one defense in this dissertation with a confirmed real effect.

`instruction_detection`'s mechanism evidence tells a different story than its significance test alone: this is underpowered, not disproven, and the  $n = 40$  cap reflects a real compute constraint, a transformer classifier run once per retrieved passage on CPU, rather than a design choice, a real-hardware timing measurement found a single item took four minutes and twenty-seven seconds wall-clock even after fixing a CPU thread-oversubscription bug. The correction does not settle this defense's headline conclusion either way: the significance test says no effect, the mechanism evidence says otherwise, and this dissertation reports both rather than favoring one.

Figure 5.8 captures the false-positive side of `instruction_detection`: it flags 4.7% of benign HotpotQA inputs, 3.3% of benign MS MARCO inputs, 0.0% of XSTest's safe prompts (Röttger et al., 2024), and 2.7% of OR-Bench's hard-but-safe prompts (Cui et al., 2025).

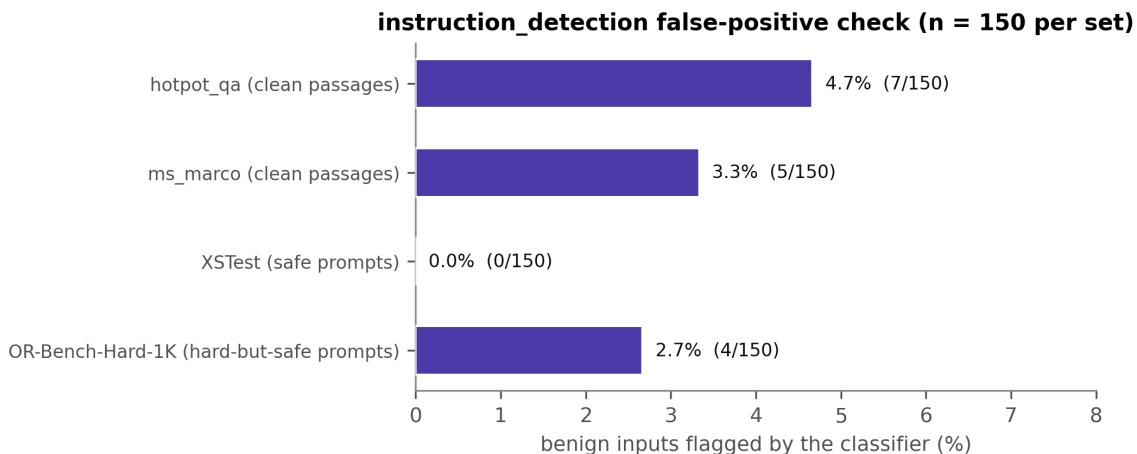


Figure 5.8: Instruction-detection false-positive rate across benign input sets.

The 0.0% result on XSTest is a favorable result in isolation, but the 4.7% rate on real corpus

passages shows that a classifier’s performance on a curated benchmark of adversarially designed safe prompts does not automatically predict its behavior on ordinary retrieved text, exactly the gap pairing two different false-positive sources was meant to expose.

## 5.10 Defense-Utility Trade-off

Table 5.7 reports mean  $\Delta F_1$  for every attack-defense combination against its matching clean baseline, across all eight model-corpus cells; Figure 5.9 visualizes the same data.

| Attack      | Defense               | Mean $\Delta F_1$    |
|-------------|-----------------------|----------------------|
| Injection   | instruction_detection | −0.0014 (negligible) |
| Injection   | output_filter         | −0.0161 (small)      |
| Injection   | spotlighting          | −0.2831 (severe)     |
| PoisonedRAG | instruction_detection | −0.3011 (severe)     |
| PoisonedRAG | output_filter         | −0.2873 (severe)     |
| PoisonedRAG | spotlighting          | −0.3067 (severe)     |

Table 5.7: Mean utility cost by attack-defense combination, all eight model-corpus cells.

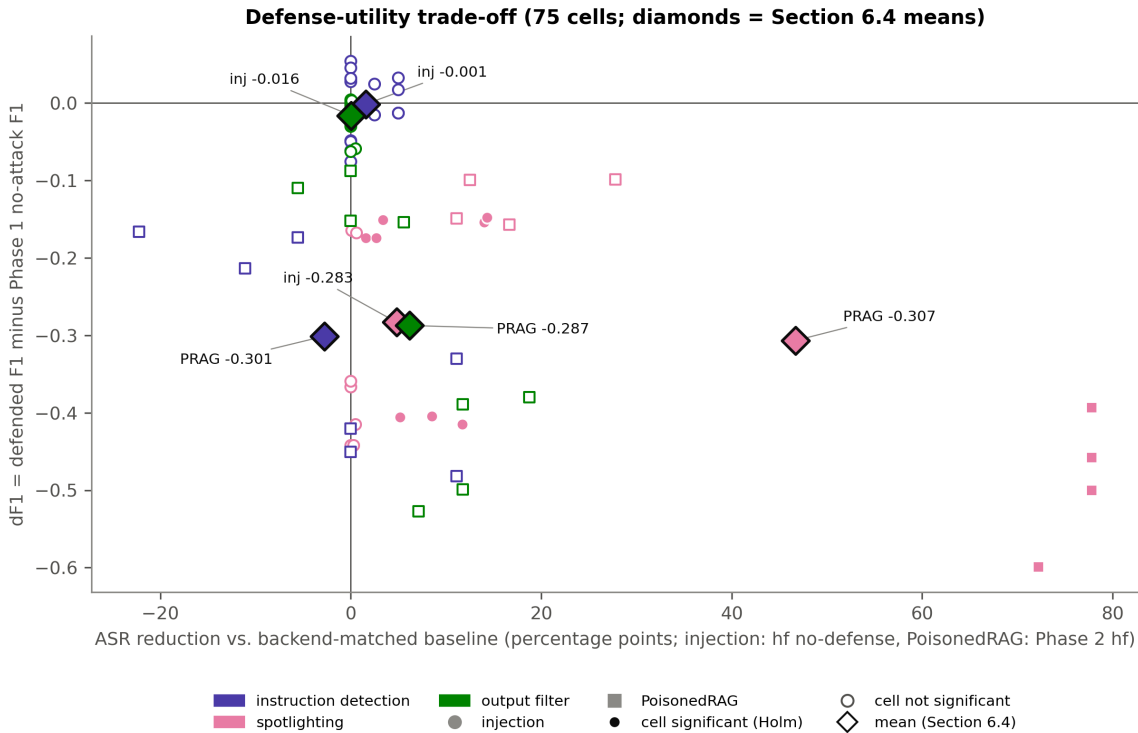


Figure 5.9: Defense effectiveness against utility cost, all three defenses.

For injection, `instruction_detection` is essentially free on utility, since it only drops passages an off-the-shelf classifier actually flags, leaving most legitimate context intact, while `spotlighting`’s encoding devastates it. For PoisonedRAG, all three defenses cost roughly the same large utility hit, none of them repair the underlying retrieval corruption, they only change what the model does with an already-poisoned context, refuse, hedge, or fail to extract an answer, which reads as a wrong answer either way against gold-answer F1. This is the central pattern this figure makes visible: reducing an attack’s success rate and preserving the system’s normal answer quality pull in opposite directions, and no defense tested here fully escapes that tension.

## 5.11 Efficiency: A Named Scope Boundary

This dissertation measures robustness only. It does not measure inference latency, throughput, memory footprint, or any other speed-versus-robustness trade-off for any of the four models or three defenses, and this is a genuine limitation of the evaluation as it stands, not an oversight to gloss over. A practitioner weighing this dissertation’s robustness findings against a real deployment decision would still need a separate efficiency evaluation this dissertation does not provide. Spotlighting’s base64 encoding, for instance, adds per-request overhead that this dissertation never measured, and this dissertation never characterized instruction-detection’s per-passage classifier cost at production scale either; Chapter 4 reports a single 4-minute-27-second wall-clock timing, collected to explain that defense’s sample-size cap, not to serve as a throughput benchmark, and it should not be read as one. This dissertation names robustness-versus-efficiency trade-off measurement explicitly as a concrete, well-motivated direction for future work, addressed further in Chapter 6, instead of folding it silently into its own robustness conclusions.

## 5.12 Comparison to Related Benchmarks

Chapter 3 introduced three comparator benchmarks against which this dissertation’s design was positioned; this section returns to two of them where a direct numerical comparison of findings, not just design scope, is possible. RSB’s own headline defense finding, that its strongest defense reduces poisoning success but at an accuracy cost exceeding 20% relative to its clean baseline in most settings, parallels this dissertation’s spotlighting result almost exactly in shape: a confirmed real effect purchased at a real, substantial utility cost, even though the two benchmarks measure entirely different attack families and use entirely different metrics. That two independent benchmarks, built by different teams on different attacks, converge on the same pattern, a real defense effect and full utility preservation essentially never arriving together, is a stronger basis for that claim than either result alone would provide. Neither SafeRAG nor Hidden-in-Plain-Text report a comparably structured defense-utility number, so no equivalent direct comparison is possible against those two; this is itself consistent with Chapter 3’s positioning of this dissertation’s design as the only one of the four to test defenses under a shared, utility-tracked, backend-controlled setup across three attack families at once.

## 5.13 Mapping to EU AI Act Article 15

This mapping is a technical exercise, testing this dissertation’s results against Article 15(5)’s named vulnerability classes, not a legal claim about any deployed system’s compliance status. Table 5.8 classifies each result explicitly rather than offering one blended verdict.

| Article 15 clause                       | Verdict                           | Basis                                                                                                                                                                                 |
|-----------------------------------------|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15(1)/(3) accuracy                      | Tested, not demonstrated          | Measured and declared per condition; "throughout lifecycle" implies deployment-drift monitoring this fixed-point benchmark does not test                                              |
| 15(4) fault-resilience                  | Not tested                        | A named gap; the engineering-bug narrative in Chapter 4 is adjacent engineering discipline, not a test of runtime fault resilience                                                    |
| 15(5) data poisoning                    | Directly tested                   | The <code>output_filter</code> structural-mismatch finding above is the concrete result: an "appropriate mitigation" failing structurally for a specific, plausible real-world choice |
| 15(5) adversarial examples              | Directly tested, nuanced          | No defense tested here achieves both a confirmed effect and utility preservation simultaneously                                                                                       |
| 15(5) model poisoning / confidentiality | Not applicable                    | All three attacks here are inference-time; none touch training data or model weights                                                                                                  |
| 15(5) general clause (Crescendo)        | Fits, with a drafting observation | Article 15(5)'s illustrative list does not name multi-turn escalation, though it plainly falls under the general vulnerability-exploitation clause                                    |

Table 5.8: Mapping of this dissertation's findings onto Article 15's clauses.

The Crescendo mapping carries one further consequence of the engineering gap already noted in Section 5.9: since `output_filter`'s guard verdict is never fed back into the conversation, the 67–82% of successful Crescendo attacks where some turn had already triggered a flag represents attacks the system could plausibly have stopped, had it been wired to act on its own diagnostic output.

The "no harmonised standard" claim running through this mapping is not a vague assertion; it has a specific, checkable status. As of the verification underlying this dissertation's own regulatory mapping, the CEN-CENELEC JTC 21 harmonised standards intended to cover Article 15's accuracy and robustness requirement, prEN 18229-2, and its cybersecurity requirement, prEN 18282, both remain in draft status and have not been published in the Official Journal. No presumption-of-conformity testing standard therefore exists yet for the exact requirement this dissertation's own methodology maps against. This dissertation does not claim to fill that standards gap; it documents, with a concrete worked example, what the gap looks like from the evaluation side, a narrower and more defensible claim than asserting this benchmark itself constitutes or anticipates the eventual standard.

## 5.14 Implications for Practitioners

Four implications follow directly from the results above, each grounded in a specific finding rather than stated as general advice. First, a practitioner should not assume a defense's effectiveness generalizes across attack types: `output_filter` appeared to strongly reduce injection ASR before backend correction, and that apparent effect collapsed to 0 of 19 cells once the confound was isolated, a direct cautionary example against trusting an uncorrected before/after comparison at face value. Second, a practitioner should match the defense to the actual threat model instead of assuming a well-built defense transfers automatically: `output_filter`'s 0.0000

flag rate against PoisonedRAG is not a tuning problem, a safety-content classifier has no mechanism to see factual poisoning by construction, and this dissertation’s data shows a defense can be competently built and still be the wrong tool for a given attack surface. Third, a practitioner should treat a small-sample result with caution: the `instruction_detection`  $n = 40$  cap reflects a real compute constraint this dissertation encountered, not a design choice, and its 0 of 16 significant result at that sample size cannot, on this dissertation’s own evidence, be distinguished from a real but underpowered effect, given that 10 of 10 backend-corrected blocks carry a genuine classifier fingerprint. Fourth, the EU AI Act Article 15 compliance-testing gap discussed above is current and unresolved, not a historical footnote, and this dissertation’s own mapping speaks to it directly instead of gesturing at it in the abstract.

## 5.15 Answering the Research Questions

RQ1 asked how vulnerable these four models are to each attack family. The answer is not a single number but a structure: vulnerability is real and substantial for every attack tested, but which model is most at risk depends entirely on which attack and, for two of the three attacks, which corpus, is in question. Ministral-3-8B is the most exposed model under injection, all four models are near-equally exposed under PoisonedRAG once corpus is held constant, and no model is reliably safer than any other under Crescendo, though the four differ sharply in the behavioral mechanism behind that shared ASR range. A single aggregate robustness score across all three attacks would average away exactly the structure this dissertation’s results say matters most.

RQ2 asked how effective practical, low-cost defenses are, and at what cost. Across three defenses tested against two of the three attack families, exactly one, spotlighting against injection, clears the bar of a confirmed, backend-isolated real effect, and it does so at a non-trivial utility cost. `output_filter` clears no such bar against either attack it was tested on. `instruction_detection`’s result is genuinely mixed: statistically unconfirmed at its available sample size, but mechanistically real. No defense tested here achieves both a confirmed effect and full utility preservation at once, which is this dissertation’s central answer to RQ2: low-cost, practical defenses in this space are not yet free, and the honest way to report a defense’s value is as a pair of numbers, effect and cost, never one number alone.

# Chapter 6

## Conclusion

### 6.1 Summary of Contributions

This dissertation set out to answer two questions. RQ1 asked how vulnerable small, open-source, instruction-tuned language models are to indirect prompt injection, knowledge poisoning, and multi-turn jailbreaking when deployed in a retrieval-augmented generation pipeline. RQ2 asked how effective practical, low-cost defenses are at reducing that vulnerability, and at what cost. Chapter 5 answered both directly: vulnerability is real and substantial across every attack tested, but its shape depends entirely on which attack and, for two of three attacks, which corpus is in question, and exactly one of three defenses tested clears the bar of a confirmed, backend-isolated real effect, at a non-trivial utility cost.

| Contribution                | Summary                                                                                                                                              |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Threat taxonomy          | Three-attack-family benchmark mapped onto Article 15(5), surfacing where the Article’s illustrative list does not cleanly name a RAG-specific threat |
| 2. Testing discipline       | Paired exact testing with Holm-Bonferroni correction, informing what adequate testing could mean under Article 15(2)                                 |
| 3. Methodological warning   | The backend-confound discovery, a citable caution against certifying a defense without holding its inference backend constant                        |
| 4. Mitigation reality check | An honest account of what three real, low-cost defenses achieve and fail to achieve against three distinct attacks                                   |

Table 6.1: This dissertation’s four contributions, summarized.

Four contributions follow from that answer, summarized in Table 6.1. First, this dissertation establishes a three-attack, four-model, backend-controlled benchmark that maps onto Article 15(5)’s illustrative vulnerability list while also surfacing where that list’s classical machine-learning framing does not cleanly cover a RAG-relevant threat: Article 15(5) names data poisoning, model poisoning, and adversarial examples explicitly, but Crescendo’s multi-turn conversational escalation fits only under the Article’s general resilience clause, not any named category. Second, it demonstrates a statistical testing discipline, paired exact tests and Holm-Bonferroni correction, that could inform what "adequate testing" should mean for Article

15(2)’s benchmark mandate, in a literature that reports significance testing inconsistently at best. Third, the backend-confound discovery, detailed in Chapter 4 and applied throughout Chapter 5, is a citable warning about a specific class of testing error a compliance regime could walk directly into: the inference stack a defense runs on must be held constant against its baseline, or a compliance test risks certifying a defense that, once the infrastructure mismatch is corrected, does nothing at all. Fourth, this dissertation offers an honest account of what current, real, off-the-shelf inference-time mitigations actually achieve against three distinct attack families: one mitigation structurally wrong for its threat class (`output_filter` against PoisonedRAG), one costing severe utility for a real effect (spotlighting), and one mechanism-confirmed but statistically underpowered (`instruction_detection`).

This dissertation does not claim more than this. It is not a legal compliance assessment, it is not a conformity assessment under Article 43, it does not establish that any specific deployed RAG system falls under Annex III’s high-risk classification, and it does not test every vulnerability class Article 15(5) names, model poisoning and confidentiality attacks are named gaps, not findings. Keeping that boundary explicit is what keeps the Article 15 mapping honest rather than overreaching beyond what three attack families and four models can actually demonstrate.

## 6.2 Practical Implications

Every claim below is scoped deliberately to what this dissertation’s own results show for the specific models, attacks, and defenses tested, never offered as a general recommendation beyond that evidence.

There is no single best model, only a robustness ranking that inverts depending on which attack is asked about. Llama-3.1-8B is this dataset’s strongest performer against injection and, at the same time, its weakest against Crescendo at 70.2%, the highest of the four. A practitioner asking "which model should I deploy" is asking an underspecified question until the threat model is named.

The defense scorecard carries three distinct verdicts, not one blended result. Spotlighting is the only defense with a confirmed, backend-isolated real effect, but that effect is more modest than its original uncorrected headline suggested, and it costs a real utility price. Output-filter’s apparent effect collapsed almost entirely once correction isolated the backend confound, and its zero flag rate against PoisonedRAG reflects a structural mismatch, not a tuning failure. Instruction-detection’s statistical result reads as null at its available sample size, while its mechanism evidence says otherwise, underpowered, not disproven.

This dissertation never measured efficiency, and that is a named scope boundary, not an oversight folded silently into the robustness findings above. Latency, throughput, and memory footprint for any of the four models or three defenses remain entirely outside this evaluation’s reach; a deployer weighing these robustness results against a real system still needs a separate efficiency study this dissertation does not provide.

Finally, these findings carry one general lesson worth stating plainly: do not assume a defense’s effectiveness generalizes across attack types, match a defense to its actual threat model rather than its reputation, and treat a small-sample statistical result with real caution before concluding a mechanism does not work. The compliance-testing gap for this kind of robustness evidence under the EU AI Act remains genuinely open today, and this dissertation’s own mapping speaks to it with a concrete worked example rather than a general assertion.

## 6.3 Limitations

The list below appears in full, not softened into vague hedges, since a reader evaluating this dissertation’s evidence needs to see exactly where that evidence is thinner than the headline results might suggest.

This dissertation’s external validity is bounded by four design choices, and none of its findings should be read as claims about LLM/RAG robustness in general. The four target models were selected against five explicit criteria, a permissive open license, self-hostable weights, instruction tuning, a 4-12B parameter range, and a dense, non-mixture-of-experts architecture, which means every result here describes small, densely-parameterized, openly licensed models and no others; whether a substantially larger model, a mixture-of-experts architecture, or a closed, API-only model would show the same vulnerability patterns, the same model-versus-corpus-versus-neither axis structure, or the same defense trade-offs is an open question this dissertation does not test. The RAG configuration itself, FAISS similarity search over sentence-transformer embeddings, top-5 retrieval, a plain retrieve-then-generate pipeline with no agentic tool use or multi-hop retrieval, is one architecture among several common ones; a different retrieval strategy, chunking approach, or embedding model could plausibly change how much surface indirect injection or PoisonedRAG actually finds to exploit. The two retrieval corpora, HotpotQA and MS MARCO, are general-domain question-answering datasets; specialized domains, medical, legal, or code-focused RAG deployments, were not tested here, and their answer structure, passage length, and factual density differ enough from these two corpora that the corpus-identity effect this dissertation found in PoisonedRAG cannot be assumed to hold, or fail to hold, in either direction on a domain this design never touched. Finally, each attack and defense is implemented as one version of a broader technique, this dissertation’s injection templates, poison-generation prompt, and escalation strategy are not the only reasonable implementations of their respective attack families, and a differently engineered version of any of them might well produce different numbers on the same four models. Taken together, these four boundaries mean this dissertation’s findings describe one benchmark’s four models, one RAG architecture, two corpora, and three attack implementations, and generalizing beyond that combination is an empirical question for future work, not a conclusion this dissertation’s own evidence supports.

The `nq_open` exclusion, confirmed and approved as a settled scope decision, still reduces this dissertation’s corpus coverage for the two corpus-dependent attack families to two corpora rather than three; Crescendo, being corpus-independent, is entirely unaffected. Ministral-3-8B’s injection figures describe genuine partial compliance with an injected instruction, not full task abandonment, a distinction this dissertation states explicitly rather than reporting an unqualified hijacking rate. The sample-size asymmetry within injection testing, 40 for instruction-detection against 1000 for the other two defenses, reflects a real per-item compute cost, a transformer classifier run once per retrieved passage on CPU, not a chosen trade-off. PoisonedRAG’s defended-condition cells drew a matched subset of 14 to 18 items per cell from a baseline of 90 to 96, after excluding items new to the defended run; no cell required an unpaired statistical fallback. Crescendo’s Phase 3 behavior pool overlaps its Phase 2 baseline only partially, 9 of 20 behaviors per model appear in both, so the usable paired sample for every Crescendo cell is 9, not 20, materially widening this dissertation’s confidence intervals on those diagnostic-only numbers. This dissertation corrected the HarmBench behavior pool’s true size from an initially assumed 510 to a verified 400 (200 standard, 100 contextual, 100 copyright) before any real sweep ran, using the gated `walledai/HarmBench` variant specifically. ASR scoring across all three attacks relies on rule-based substring matching, which this dissertation confirms conflates genuinely different phenomena, verbatim echo, marker-glued-to-a-correct-answer, and reasoned compliance for injection, under one binary success flag. This dissertation individually audited and confirmed the

refusal-detection function’s false-positive rate only for Ministral-3-8B; it plausibly affects an unquantified fraction of the other three models’ flagged rows, each in the 23-to-44 range, never individually audited here. This dissertation used a single attacker-and-judge model, DeepSeek V4 Pro, and a single behavior-pool composition for Crescendo, so it cannot distinguish "these four models resist this style of escalation similarly" from "this specific attacker’s strategy happens to land similarly on these four targets." It used a single poison configuration and a single poison-generator model for PoisonedRAG, so it likewise cannot distinguish a corpus’s genuine resistance from an artifact of that specific generator. This dissertation verified the atomic-write guarantee underlying its cache-integrity design only on a standard Linux filesystem; the project never provisioned an originally planned Azure Files network mount before migrating to RunPod, so no verification gap remains open on the infrastructure actually used. Finally, this dissertation never exercised the unpaired statistical fallback built into its testing code against real data, since every one of the 79 Phase 3 cells had a non-empty matched baseline subset; that code path exists but remains untested here.

## 6.4 Future Work

Three directions follow directly from gaps this dissertation names explicitly, distinct from the limitations above, since these are unfinished engineering and unexplored scope rather than weaknesses in work already completed.

The output-filter defense logs its guard verdict against Crescendo but deliberately never wires that verdict to enforce anything live in the conversation; converting its 67-to-82% counterfactual catch rate into a measured ASR reduction, substituting a refusal marker into the live conversation history the same way the injection and PoisonedRAG runners already do, is this dissertation’s clearest, most concrete next step. A robustness-versus-efficiency trade-off study, named but not attempted here, would measure the latency, throughput, and memory cost each defense adds against the robustness gain it provides, closing the scope boundary Chapter 5 names explicitly. Finally, model poisoning and confidentiality attacks, the two vulnerability classes Article 15(5) names that this dissertation does not test, are a natural extension of this dissertation’s own threat taxonomy, since both operate on the model or its training data rather than at inference time, an entirely different attack surface from all three families studied here.

# References

- Abouelenin, A. et al. (2025). Phi-4-Mini technical report: Compact yet powerful multimodal language models via mixture-of-LoRAs. arXiv:2503.01743. <https://arxiv.org/abs/2503.01743>. Technical report, not peer-reviewed. Model: <https://huggingface.co/microsoft/Phi-4-mini-instruct>.
- Chao, P., Debenedetti, E., Robey, A., Andriushchenko, M., Croce, F., Schwag, V., Dobriban, E., Flammarion, N., Pappas, G. J., Tramèr, F., Hassani, H., and Wong, E. (2024). JailbreakBench: An open robustness benchmark for jailbreaking large language models. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track*. [https://proceedings.neurips.cc/paper\\_files/paper/2024/hash/63092d79154adebd7305dfd498cbff70-Abstract.html](https://proceedings.neurips.cc/paper_files/paper/2024/hash/63092d79154adebd7305dfd498cbff70-Abstract.html). Source of JBB-Behaviors, 20% of the stratified behavior pool.
- Cui, J., Chiang, W.-L., Stoica, I., and Hsieh, C.-J. (2025). OR-Bench: An over-refusal benchmark for large language models. In *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025)*, volume 267 of *Proceedings of Machine Learning Research*, pages 11515–11542. PMLR. <https://proceedings.mlr.press/v267/cui25a.html>.
- DeepSeek-AI (2026). DeepSeek-V4: Towards highly efficient million-token context intelligence. arXiv:2606.19348. <https://arxiv.org/abs/2606.19348>. Technical report, not peer-reviewed. Used as Crescendo’s attacker and judge model, never a target.
- Efron, B. (1979). Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26. <https://doi.org/10.1214/aos/1176344552>.
- Fisher, R. A. (1922). On the interpretation of  $\chi^2$  from contingency tables, and the calculation of p. *Journal of the Royal Statistical Society*, 85(1):87–94. <https://doi.org/10.2307/2340521>.
- Grattafiori, A. et al. (2024). The Llama 3 herd of models. arXiv:2407.21783. <https://arxiv.org/abs/2407.21783>. Technical report, not peer-reviewed. Model: <https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct>.
- Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., and Fritz, M. (2023). Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec ’23)*, pages 79–90. Association for Computing Machinery. <https://doi.org/10.1145/3605764.3623985>. Foundational paper establishing indirect prompt injection as an attack class.
- Guo, H. and Wei, Z. (2026). Hidden-in-plain-text: A benchmark for social-web indirect prompt injection in RAG. In *Proceedings of the ACM Web Conference 2026 (WWW ’26)*, Dubai, United Arab Emirates. Association for Computing Machinery. <https://doi.org/10.1145/3774904.3792853>. Named competitor benchmark (Meeting 4, supervisor-confirmed) – scoped to web-carrier indirect prompt injection only, not knowledge poisoning or multi-turn jailbreaking.

- He, P., Gao, J., and Chen, W. (2023). DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing. In *Proceedings of the Eleventh International Conference on Learning Representations (ICLR 2023)*. <https://openreview.net/forum?id=sE7-XhLxHA>. Architecture underlying the fine-tuned classifier used for the instruction-detection defense.
- Hines, K., Lopez, G., Hall, M., Zarfati, F., Zunger, Y., and Kiciman, E. (2024). Defending against indirect prompt injection attacks with spotlighting. In *Proceedings of the Conference on Applied Machine Learning for Information Security (CAMLIS 2024)*, volume 3920 of *CEUR Workshop Proceedings*. <https://ceur-ws.org/Vol-3920/paper03.pdf>. The defense implemented directly in this thesis; encoding-mode variant specifically evaluated.
- Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70.
- Hugging Face (2016). Hugging face – the AI community building the future. <https://huggingface.co>. Model hosting and inference platform; hosts every model weight and dataset used in this project. Accessed September 2026.
- Johnson, J., Douze, M., and Jégou, H. (2021). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547. <https://doi.org/10.1109/TBDATA.2019.2921572>. FAISS, used for dense retrieval indexing.
- Kwiatkowski, T., Palomaki, J., Redfield, O., Collins, M., Parikh, A., Alberti, C., Epstein, D., Polosukhin, I., Devlin, J., Lee, K., Toutanova, K., Jones, L., Kelcey, M., Chang, M.-W., Dai, A. M., Uszkoreit, J., Le, Q., and Petrov, S. (2019). Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466. [https://doi.org/10.1162/tacl\\_a\\_00276](https://doi.org/10.1162/tacl_a_00276). Source of `nq_open`; supports the corpus-construction finding behind its exclusion from real sweeps.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP ’23)*, pages 611–626. Association for Computing Machinery. <https://doi.org/10.1145/3600006.3613165>. vLLM, the inference engine used throughout the sweep harness.
- Liang, X., Niu, S., Li, Z., Zhang, S., Wang, H., Xiong, F., Fan, Z., Tang, B., Zhao, J., Yang, J., Song, S., and Wang, M. (2025). SafeRAG: Benchmarking security in retrieval-augmented generation of large language model. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4609–4631, Vienna, Austria. Association for Computational Linguistics. <https://aclanthology.org/2025.acl-long.230/>. Named competitor benchmark (Meeting 4, supervisor-confirmed) – a broad four-task RAG security taxonomy, not the same three-attack-family, four-model design this thesis uses.
- Liu, Y., Jia, Y., Geng, R., Jia, J., and Gong, N. Z. (2024). Formalizing and benchmarking prompt injection attacks and defenses. In *Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24)*, pages 1831–1847. USENIX Association. <https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei>.
- Liu, Y., Jia, Y., Jia, J., Song, D., and Gong, N. Z. (2025). DataSentinel: A game-theoretic detection of prompt injection attacks. In *2025 IEEE Symposium on Security and Privacy (S&P)*, pages 2190–2208. IEEE. <https://doi.org/10.1109/SP61157.2025.00119>. Closest

peer-reviewed comparator for instruction-detection-style defenses; a different classifier was used in this thesis for practical reasons.

- Mazeika, M., Phan, L., Yin, X., Zou, A., Wang, Z., Mu, N., Sakhaee, E., Li, N., Basart, S., Li, B., Forsyth, D., and Hendrycks, D. (2024). HarmBench: A standardized evaluation framework for automated red teaming and robust refusal. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*, volume 235 of *Proceedings of Machine Learning Research*, pages 35181–35224. PMLR. <https://proceedings.mlr.press/v235/mazeika24a.html>. Source of HarmBench behaviors, 80% of the stratified behavior pool.
- McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157. <https://doi.org/10.1007/BF02295996>.
- Meta AI (2025). Llama Guard 4 model card. <https://huggingface.co/meta-llama/Llama-Guard-4-12B>. Model artifact, not a peer-reviewed paper; cite as a released model, not a study. Used for the output-filter defense.
- Mistral AI (2025). Introducing Mistral 3. <https://mistral.ai/news/mistral-3/>. No technical report exists for this release; documented via announcement blog post and model card only. Model: <https://huggingface.co/mistralai/Mistral-3-8B-Instruct-2512>.
- Moradi, R., Alizadeh Noughabi, H., Zarrinkalam, F., and Dehghantanha, A. (2026). Defending RAG against knowledge poisoning using cross-encoder activation signals. In *Proceedings of the 39th Canadian Conference on Artificial Intelligence*, volume 318 of *Proceedings of Machine Learning Research*, pages 366–376. PMLR. <https://proceedings.mlr.press/v318/moradi26a.html>. 2026 follow-on defense work; confirms PoisonedRAG remains the field’s reference threat model.
- Nakka, K. and Saxena, N. (2026). BitBypass: A new direction in jailbreaking aligned large language models with bitstream camouflage. In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 3808–3834, Rabat, Morocco. Association for Computational Linguistics. <https://aclanthology.org/2026.findings-eacl.198/>. 2026 comparison work; confirms jailbreak research remains active and currently publishing, contemporaneous with this thesis.
- Nguyen, T., Rosenberg, M., Song, X., Gao, J., Tiwary, S., Majumder, R., and Deng, L. (2016). MS MARCO: A human generated machine reading comprehension dataset. In *Proceedings of the Workshop on Cognitive Computation: Integrating Neural and Symbolic Approaches 2016, co-located with NIPS 2016*, volume 1773 of *CEUR Workshop Proceedings*. [https://ceur-ws.org/Vol-1773/CoCoNIPS\\_2016\\_paper9.pdf](https://ceur-ws.org/Vol-1773/CoCoNIPS_2016_paper9.pdf).
- NVIDIA (2024). NVIDIA NIM: Optimized inference microservices. <https://build.nvidia.com>. API platform used to access Nemotron-3-Ultra for PoisonedRAG’s poison-passage generation. Accessed September 2026.
- NVIDIA (2025). NVIDIA Nemotron 3 Ultra: Open, efficient mixture-of-experts hybrid Mamba-transformer model for agentic reasoning. <https://research.nvidia.com/labs/nemotron/files/NVIDIA-Nemotron-3-Ultra-Technical-Report.pdf>. Company-published technical report, not a peer-reviewed venue. Used as PoisonedRAG’s poison-passage generator. Model card: <https://build.nvidia.com/nvidia/nemotron-3-ultra-550b-a55b/modelcard>.

- OpenRouter (2024). OpenRouter: A unified interface for LLMs. <https://openrouter.ai>. API routing platform used to access DeepSeek-V4 as Crescendo’s attacker and judge model. Accessed September 2026.
- Protect AI (2024). `deberta-v3-base-prompt-injection-v2` model card. <https://huggingface.co/protectai/deberta-v3-base-prompt-injection-v2>. Industry-released classifier fine-tuned from the peer-reviewed DeBERTaV3 architecture (He et al., ICLR 2023); no accompanying paper.
- Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992. Association for Computational Linguistics. <https://aclanthology.org/D19-1410/>. Foundational architecture underlying `sentence-transformers/all-mpnet-base-v2`, the embedder used to build the FAISS index for every corpus.
- Röttger, P., Kirk, H. R., Vidgen, B., Attanasio, G., Bianchi, F., and Hovy, D. (2024). XSTest: A test suite for identifying exaggerated safety behaviours in large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL 2024)*. Association for Computational Linguistics. <https://aclanthology.org/2024.naacl-long.301/>. Hub handle retired; current handle is `Paul/XSTest` (same 450-prompt suite).
- RunPod (2024). RunPod: Cloud GPU rental for AI and ML. <https://runpod.io>. Cloud GPU infrastructure (A100 SXM 80GB pods) used for every real-hardware experiment in this thesis. Accessed September 2026.
- Russinovich, M., Salem, A., and Eldan, R. (2025). Great, now write an article about that: The Crescendo multi-turn LLM jailbreak attack. In *Proceedings of the 34th USENIX Security Symposium (USENIX Security 25)*. USENIX Association. <https://www.usenix.org/conference/usenixsecurity25/presentation/russinovich>. The attack implemented directly in this thesis.
- Singh, S. (2026). `rag-adversarial-robustness`: Cross-model benchmark of prompt injection, knowledge poisoning, and multi-turn attacks under EU AI Act article 15. <https://github.com/Simarjit1303/rag-adversarial-robustness>. This project’s own implementation, the technical contribution required by the CDS module handbook (Section 8). Self-citation of the author’s own artifact, not external literature.
- Song, K., Tan, X., Qin, T., Lu, J., and Liu, T.-Y. (2020). MPNet: Masked and permuted pre-training for language understanding. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*. <https://proceedings.neurips.cc/paper/2020/hash/c3a690be93aa602ee2dc0ccab5b7b67e-Abstract.html>. Pre-training objective underlying the base encoder of `all-mpnet-base-v2`.
- Yang, A. et al. (2025). Qwen3 technical report. arXiv:2505.09388. <https://arxiv.org/abs/2505.09388>. Technical report, not peer-reviewed. Model: <https://huggingface.co/Qwen/Qwen3-8B>.
- Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W. W., Salakhutdinov, R., and Manning, C. D. (2018). HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Lan-*

- guage Processing (EMNLP)*, pages 2369–2380. Association for Computational Linguistics. <https://aclanthology.org/D18-1259/>.
- Zhan, Q., Fang, R., Bindu, R., Gupta, A., Hashimoto, T., and Kang, D. (2024). InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 10471–10506. Association for Computational Linguistics. <https://aclanthology.org/2024.findings-acl.624/>. Related benchmarking work, context/discussion only.
- Zhang, B., Xin, H., Li, J., Zhang, D., Fang, M., Liu, Z., Nie, L., and Liu, Z. (2025). Benchmarking poisoning attacks against retrieval-augmented generation. arXiv:2505.18543. <https://arxiv.org/abs/2505.18543>. Named competitor benchmark (Meeting 4, supervisor-confirmed) – checked against multiple 2025-2026 papers citing it, all as an unpublished preprint; no accepted-venue version found as of this writing. Cited here as a preprint exception, not as peer-reviewed literature.
- Zou, W., Geng, R., Wang, B., and Jia, J. (2025). PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models. In *Proceedings of the 34th USENIX Security Symposium (USENIX Security 25)*, pages 3827–3844. USENIX Association. <https://www.usenix.org/conference/usenixsecurity25/presentation/zou-poisonedrag>. The attack implemented directly in this thesis.

# Appendices

## Full Baseline Results Table

Chapter 5’s Figure 5.1 summarizes Phase 1 baseline answer quality visually; Table 6.2 reproduces the complete underlying numbers for all four models across all three corpora, including **nq\_open**, retained here for completeness even though Chapter 5 excludes it from every comparison.

| Model         | Corpus                      | F1 (clean) | Exact Match | Contains Answer |
|---------------|-----------------------------|------------|-------------|-----------------|
| Llama-3.1-8B  | HotpotQA                    | 0.6058     | 0.531       | 0.562           |
| Llama-3.1-8B  | MS MARCO                    | 0.2340     | 0.092       | 0.108           |
| Llama-3.1-8B  | <b>nq_open</b> <sup>†</sup> | 0.9608     | 0.909       | 0.989           |
| Minstral-3-8B | HotpotQA                    | 0.6392     | 0.560       | 0.601           |
| Minstral-3-8B | MS MARCO                    | 0.2572     | 0.088       | 0.123           |
| Minstral-3-8B | <b>nq_open</b> <sup>†</sup> | 0.9599     | 0.910       | 0.993           |
| Phi-4-mini    | HotpotQA                    | 0.4600     | 0.371       | 0.471           |
| Phi-4-mini    | MS MARCO                    | 0.2539     | 0.074       | 0.146           |
| Phi-4-mini    | <b>nq_open</b> <sup>†</sup> | 0.9649     | 0.923       | 0.984           |
| Qwen3-8B      | HotpotQA                    | 0.6363     | 0.557       | 0.599           |
| Qwen3-8B      | MS MARCO                    | 0.2755     | 0.112       | 0.166           |
| Qwen3-8B      | <b>nq_open</b> <sup>†</sup> | 0.9623     | 0.921       | 0.991           |

Table 6.2: Complete Phase 1 baseline results, all model-corpus pairs,  $n = 1000$  each.

*Note.* <sup>†</sup>**nq\_open** rows are retained for completeness only; Chapter 5 properly documents and excludes this corpus from every comparison.

## Mechanism Attribution Table

Table 6.3 reproduces, in full, the mechanism evidence Chapter 5 cites narratively when distinguishing a statistically significant effect from a mechanistically real one.

| Defense               | Attack      | Mechanism Metric                                                       | Result                       |
|-----------------------|-------------|------------------------------------------------------------------------|------------------------------|
| output_filter         | Injection   | Guard-caught fraction of blocked items                                 | 12/1,511 (0.8%)              |
| output_filter         | PoisonedRAG | Guard flag rate, all scored responses                                  | 0/160 (0.0000)               |
| output_filter         | Crescendo   | Fraction of unfiltered successes the guard would have flagged          | 67–82% (counterfactual only) |
| instruction_detection | Injection   | Fraction of backend-corrected blocks with at least one flagged passage | 10/10 (100%)                 |

Table 6.3: Mechanism attribution across all three defenses, full underlying figures.

## List of Abbreviations

| Abbreviation     | Full Term                                      |
|------------------|------------------------------------------------|
| AI               | Artificial Intelligence                        |
| API              | Application Programming Interface              |
| ASR              | Attack Success Rate                            |
| CI (statistics)  | Confidence Interval                            |
| CI (engineering) | Continuous Integration                         |
| CI/CD            | Continuous Integration / Continuous Deployment |
| CPU              | Central Processing Unit                        |
| CSV              | Comma-Separated Values                         |
| EU               | European Union                                 |
| FAISS            | Facebook AI Similarity Search                  |
| GHCR             | GitHub Container Registry                      |
| GPU              | Graphics Processing Unit                       |
| HTTP             | Hypertext Transfer Protocol                    |
| JSONL            | JSON Lines                                     |
| LLM              | Large Language Model                           |
| ML               | Machine Learning                               |
| MSc              | Master of Science                              |
| RAG              | Retrieval-Augmented Generation                 |
| SDK              | Software Development Kit                       |
| SSH              | Secure Shell                                   |

# List of Figures

|     |                                                                                                                                                          |    |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1 | Structure of the dissertation, chapters in reading order. . . . .                                                                                        | 4  |
| 1.2 | Conceptual framework: how attacks, defenses, and outcomes connect through this dissertation’s statistical methodology to its Article 15 mapping. . . . . | 5  |
| 2.1 | The retrieve-then-generate pipeline used throughout this dissertation. . . . .                                                                           | 6  |
| 3.1 | Literature landscape: sources grouped by attack family and defense mechanism.                                                                            | 12 |
| 4.1 | Operational pipeline: from a GitHub push to a verified, committed result. . . . .                                                                        | 14 |
| 4.2 | PoisonedRAG attack mechanism as implemented in this dissertation. . . . .                                                                                | 17 |
| 4.3 | Crescendo multi-turn jailbreak mechanism as implemented in this dissertation. .                                                                          | 18 |
| 4.4 | General backend-confound correction pattern, applied per defense-attack combination. . . . .                                                             | 21 |
| 5.1 | Phase 1 baseline answer quality (F1, exact-match) by model and corpus. . . . .                                                                           | 23 |
| 5.2 | Evidence for the <b>nq_open</b> exclusion: near-ceiling, near-invariant F1 across all four models. . . . .                                               | 24 |
| 5.3 | Indirect injection ASR by model and template. . . . .                                                                                                    | 25 |
| 5.4 | PoisonedRAG ASR by model and corpus. . . . .                                                                                                             | 27 |
| 5.5 | Crescendo ASR by model, with bootstrap confidence intervals. . . . .                                                                                     | 28 |
| 5.6 | Backend-confound correction: significant cells before and after, by defense. . . .                                                                       | 31 |
| 5.7 | Backend-confound verdicts across defense-attack combinations. . . . .                                                                                    | 31 |
| 5.8 | Instruction-detection false-positive rate across benign input sets. . . . .                                                                              | 32 |
| 5.9 | Defense effectiveness against utility cost, all three defenses. . . . .                                                                                  | 33 |

# List of Tables

|     |                                                                                                                                 |    |
|-----|---------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1 | Core RAG pipeline components used in this benchmark. . . . .                                                                    | 6  |
| 3.1 | Positioning this dissertation against its three named comparator benchmarks. . .                                                | 13 |
| 4.1 | The four target language models benchmarked in this dissertation. . . . .                                                       | 15 |
| 5.1 | Indirect injection ASR (%) by model, corpus, and template, $n = 1000$ per cell, 40 cells total. . . . .                         | 24 |
| 5.2 | PoisonedRAG full per-cell results: attack success, retrieval-echo rate, and utility drop with 95% confidence intervals. . . . . | 26 |
| 5.3 | Crescendo full per-model results: attack success rate against refusal and back-track behavior. . . . .                          | 28 |
| 5.4 | Three attacks, three different dominant axes. . . . .                                                                           | 29 |
| 5.5 | Robustness profile by model, all three attacks. No model is uniformly strongest or weakest. . . . .                             | 30 |
| 5.6 | Backend-confound correction outcomes across defense-attack combinations. . . .                                                  | 30 |
| 5.7 | Mean utility cost by attack-defense combination, all eight model-corpus cells. . .                                              | 33 |
| 5.8 | Mapping of this dissertation’s findings onto Article 15’s clauses. . . . .                                                      | 35 |
| 6.1 | This dissertation’s four contributions, summarized. . . . .                                                                     | 37 |
| 6.2 | Complete Phase 1 baseline results, all model-corpus pairs, $n = 1000$ each. . . .                                               | 46 |
| 6.3 | Mechanism attribution across all three defenses, full underlying figures. . . . .                                               | 47 |